

# SQL Server 2025 — Apuntes, funciones y ejercicios resueltos

---

## Índice

0. Si me quedo en blanco: protocolo
  1. Conceptos y código
  2. Diccionario de funciones
  3. Otros comandos útiles (conjuntos, TOP, INSERT/UPDATE/DELETE, DDL...)
  4. Ejercicios resueltos y explicados
  5. Chuleta rápida
  6. Errores típicos y cómo arreglarlos
  7. Estrategia y checklist final
- 

## 0. Si me quedo en blanco: protocolo

---

Respira y sigue estos pasos **en orden**. No hace falta saber la solución completa: hay que avanzar un paso cada vez.

### Paso 1 — Subraya en el enunciado

- ¿Qué COLUMNAS piden y con qué NOMBRE (alias)?
- ¿Qué FILTROS hay? (fechas, país, importe mínimo...)
- ¿Qué ORDEN piden?
- ¿Imponen una TÉCNICA? ("CTE", "sin JOIN", "SELECT INTO", "vista", "temporal"...)
- ¿Piden "todos aunque no tengan..."? → LEFT JOIN

### Paso 2 — Dibuja el camino entre tablas

Ejemplo: cobros → facturas → clientes → empleados  
Para cada flecha: ¿qué columna las une? (cobros.factura\_id = facturas.factura\_id)

### Paso 3 — Escribe el esqueleto

```
SELECT  <columnas>
FROM    <tabla principal>
JOIN    <otras tablas> ON ...
WHERE   <filtros de FILAS>
GROUP BY <columnas no agregadas>
```

```
HAVING <filtros de GRUPOS>
ORDER BY <orden>;
```

## Paso 4 — Construye por capas y prueba cada capa

```
-- 1) Solo el FROM y los JOIN
SELECT TOP 10 * FROM facturas f JOIN clientes c ON c.cliente_id = f.cliente_id;
-- 2) Añade WHERE
-- 3) Añade GROUP BY / agregados
-- 4) Añade ORDER BY
```

## Paso 5 — Decide la técnica con esta tabla

| Si el enunciado dice...                              | Usa...                               |
|------------------------------------------------------|--------------------------------------|
| "combina", "cruza", "con su cliente"                 | INNER JOIN                           |
| "todos los clientes aunque no tengan..."             | LEFT JOIN                            |
| "sin facturas", "que no tienen..."                   | NOT EXISTS (o LEFT JOIN ... IS NULL) |
| "que tienen al menos un..."                          | EXISTS                               |
| "mayor que la media"                                 | subconsulta escalar                  |
| "pertenece a...", "entre los clientes de..."         | IN                                   |
| "paso intermedio", "reutilizar"                      | CTE                                  |
| "directa o indirectamente", "niveles", "jerarquía"   | CTE recursiva                        |
| "para que otros la consulten sin conocer las tablas" | VIEW                                 |
| "foto", "copia estática", "que no cambie"            | SELECT INTO (tabla física)           |
| "solo en mi sesión", "paso previo de trabajo"        | #temporal                            |
| "clasificar", "estado", "etiqueta"                   | CASE                                 |
| "si no hay dato, pon 0"                              | COALESCE                             |
| "ranking", "top N por grupo"                         | ROW_NUMBER() OVER (...)              |

## Plantilla de consulta completa (copiar y adaptar)

```
WITH paso1 AS (
    SELECT ...
    FROM ...
    WHERE ...
    GROUP BY ...
),
paso2 AS (
```

```
SELECT ...  
FROM paso1  
JOIN ...  
)  
SELECT  
    t.col1,  
    t.col2 AS alias,  
    COALESCE(SUM(x.importe), 0) AS total  
FROM tabla t  
LEFT JOIN otra x  
    ON x.id = t.id  
WHERE t.fecha >= '2026-07-01'  
    AND t.fecha <= '2026-09-30'  
GROUP BY t.col1, t.col2  
HAVING SUM(x.importe) > 1000  
ORDER BY total DESC;
```

# 1. Conceptos y código

## 1.0. Orden lógico de ejecución de una consulta

**Esto explica casi todos los errores.** SQL no ejecuta en el orden en que lo escribes:

|             |                |                                     |
|-------------|----------------|-------------------------------------|
| Se escribe: | Se ejecuta:    |                                     |
| SELECT      | 5. SELECT      | (aquí nacen los alias)              |
| FROM        | 1. FROM / JOIN |                                     |
| WHERE       | 2. WHERE       | (filtra FILAS, antes de agrupar)    |
| GROUP BY    | 3. GROUP BY    |                                     |
| HAVING      | 4. HAVING      | (filtra GRUPOS, después de agrupar) |
| ORDER BY    | 6. ORDER BY    | (aquí SÍ se pueden usar alias)      |

Consecuencias:

| Regla                                                                                    | Motivo                                  |
|------------------------------------------------------------------------------------------|-----------------------------------------|
| No puedes usar un <b>alias del SELECT en el WHERE</b>                                    | el WHERE se ejecuta antes que el SELECT |
| Puedes usar el alias en el <b>ORDER BY</b>                                               | el ORDER BY se ejecuta después          |
| No puedes usar <b>SUM()</b> , <b>COUNT()</b> ... en el <b>WHERE</b>                      | aún no hay grupos → usa <b>HAVING</b>   |
| Un filtro sobre una fila ( <b>pais = 'ES'</b> ) va en <b>WHERE</b> , no en <b>HAVING</b> | es más claro y más rápido               |

## 1.1. JOIN

Un **JOIN** permite combinar información de varias tablas utilizando una relación entre ellas.

## INNER JOIN

Devuelve únicamente las filas que tienen coincidencia en ambas tablas.

```
SELECT *  
FROM clientes c  
INNER JOIN facturas f  
    ON f.cliente_id = c.cliente_id;
```

**Idea:**

Diagrama que ilustra el INNER JOIN. Se muestran dos tablas, 'clientes' y 'facturas', con flechas descendentes que indican sus respectivos conjuntos de datos. Una línea horizontal con el texto 'coinciden' en el centro conecta las flechas, representando la intersección de los datos de ambas tablas.

Solo aparecen clientes que tienen facturas.

---

## LEFT JOIN

Conserva **todas las filas de la tabla de la izquierda**.

```
SELECT *  
FROM clientes c  
LEFT JOIN facturas f  
    ON f.cliente_id = c.cliente_id;
```

Aquí:

- Todos los **clientes** aparecen.
- Si tienen facturas, se añaden.
- Si no tienen factura, las columnas de **facturas** aparecen como **NULL**.

Regla para memorizar

A LEFT JOIN B

→ **Todos los de A**, y los de B cuando coincidan.

Por eso:

```
FROM clientes c
LEFT JOIN facturas f
  ON f.cliente_id = c.cliente_id
```

se queda con **todos los clientes**, no solo con los que tienen factura.

⚠ Trampa clásica: filtro en **ON** o en **WHERE** con LEFT JOIN

```
-- ❌ MAL: el WHERE elimina las filas con NULL → el LEFT JOIN se convierte en
INNER JOIN
SELECT c.razon_social, SUM(co.importe)
FROM clientes c
LEFT JOIN cobros co ON co.cliente_id = c.cliente_id
WHERE co.fecha_cobro <= '2026-09-30'      -- los clientes sin cobros desaparecen
GROUP BY c.razon_social;

-- ✅ BIEN: el filtro de la tabla derecha va en el ON
SELECT c.razon_social, SUM(co.importe)
FROM clientes c
LEFT JOIN cobros co
  ON co.cliente_id = c.cliente_id
  AND co.fecha_cobro <= '2026-09-30'      -- los clientes sin cobros se
mantienen
GROUP BY c.razon_social;
```

Esto es exactamente lo que se hace en la **Pregunta 15**.

## Otros tipos de JOIN

| JOIN            | Qué devuelve                                          | Ejemplo                                                  |
|-----------------|-------------------------------------------------------|----------------------------------------------------------|
| INNER JOIN      | Solo coincidencias                                    | FROM A INNER JOIN B ON A.id = B.id                       |
| LEFT JOIN       | Todo A + coincidencias de B (si no, NULL)             | FROM A LEFT JOIN B ON ...                                |
| RIGHT JOIN      | Todo B + coincidencias de A                           | FROM A RIGHT JOIN B ON ... (equivale a invertir el LEFT) |
| FULL OUTER JOIN | Todo A y todo B, con NULL donde no coinciden          | FROM A FULL OUTER JOIN B ON ...                          |
| CROSS JOIN      | Producto cartesiano (todas las combinaciones, sin ON) | FROM A CROSS JOIN B                                      |
| Self join       | Una tabla contra sí misma (con alias distintos)       | ver abajo                                                |

```
-- Self join: cada empleado con el nombre de su jefe
SELECT e.nombre AS empleado, j.nombre AS jefe
FROM empleados e
LEFT JOIN empleados j
    ON j.empleado_id = e.jefe_id;
```

Patrón anti-join (encontrar los que NO tienen)

```
-- Clientes sin facturas con LEFT JOIN
SELECT c.*
FROM clientes c
LEFT JOIN facturas f
    ON f.cliente_id = c.cliente_id
WHERE f.factura_id IS NULL;      -- NULL en la clave de la tabla derecha = no hay
                                coincidencia
```

Es equivalente a **NOT EXISTS** (sección 1.6).

---

## 1.2. Subconsulta escalar

Una subconsulta escalar devuelve **un único valor** (una fila, una columna).

Ejemplo:

```
SELECT *
FROM facturas
WHERE importe > (
    SELECT AVG(importe)
    FROM facturas
);
```

La subconsulta:

```
SELECT AVG(importe)
FROM facturas
```

calcula un único valor y la consulta exterior lo utiliza.

Estructura

```
WHERE columna > (
    SELECT funcion(...)
```

```
FROM tabla  
)
```

## Dónde se puede usar una subconsulta escalar

```
-- En el WHERE (lo más habitual)  
WHERE importe > (SELECT AVG(importe) FROM facturas)  
  
-- En el SELECT (como una columna más)  
SELECT factura_id, importe,  
       importe - (SELECT AVG(importe) FROM facturas) AS dif_media  
FROM facturas;
```

Si la subconsulta devuelve **más de una fila** con un operador `=`, `>`, `<`... da error: *"Subquery returned more than 1 value"*. En ese caso usa `IN`.

## 1.3. Subconsulta con IN

`IN` sirve para comprobar si un valor pertenece al resultado de una subconsulta (o de una lista fija).

```
SELECT *  
FROM facturas  
WHERE cliente_id IN (  
    SELECT cliente_id  
    FROM clientes  
    WHERE pais = 'ES'  
);
```

Se puede leer:

Dame las facturas cuyo `cliente_id` esté entre los clientes españoles.

Con lista fija:

```
WHERE pais IN ('ES', 'FR', 'PT')  
WHERE pais NOT IN ('ES', 'FR', 'PT')
```

⚠ `NOT IN (subconsulta)` falla silenciosamente si la subconsulta devuelve algún `NULL` (no devuelve ninguna fila). Para "los que no están" usa `NOT EXISTS`.

## 1.4. Subconsultas anidadas

Una subconsulta puede contener otra subconsulta.

```
SELECT *
FROM cobros
WHERE factura_id IN (
  SELECT factura_id
  FROM facturas
  WHERE cliente_id IN (
    SELECT cliente_id
    FROM clientes
    WHERE gestor_id IN (
      SELECT empleado_id
      FROM empleados
      WHERE puesto = 'Comercial'
    )
  )
);
```

Aquí tenemos varios niveles:

```
cobros
  ↓
facturas
  ↓
clientes
  ↓
empleados
```

**Truco para escribirlas:** empieza por la **más interna** (la del final de la cadena) y ve construyendo hacia fuera.

---

## 1.5. EXISTS

**EXISTS** comprueba si una subconsulta devuelve **al menos una fila**.

```
SELECT *
FROM clientes c
WHERE EXISTS (
  SELECT 1
  FROM facturas f
  WHERE f.cliente_id = c.cliente_id
);
```

No importa qué devuelve el **SELECT** interno. Solo importa si existe alguna fila.

Por eso es habitual utilizar:

```
SELECT 1
```

La subconsulta es **correlacionada**: usa `c.cliente_id` de la consulta exterior, así que se evalúa para cada cliente.

---

## 1.6. NOT EXISTS

Es lo contrario:

```
SELECT *
FROM clientes c
WHERE NOT EXISTS (
    SELECT 1
    FROM facturas f
    WHERE f.cliente_id = c.cliente_id
);
```

Devuelve clientes **sin facturas**.

---

## 1.7. CTE

Una CTE permite crear temporalmente un resultado con nombre para utilizarlo en la consulta siguiente.

```
WITH cobros_por_factura AS (
    SELECT
        factura_id,
        SUM(importe) AS cobrado
    FROM cobros
    GROUP BY factura_id
)
SELECT *
FROM cobros_por_factura;
```

La CTE solo existe durante esa consulta.

Estructura

```
WITH nombre_cte AS (
    SELECT ...
)
SELECT ...
FROM nombre_cte;
```

## Reglas importantes

- Si hay una instrucción anterior en el mismo lote, debe terminar en `;` antes del `WITH` (o escribe `;``WITH`).
- Una CTE **solo se puede usar en la instrucción que la sigue inmediatamente**.
- No se puede poner `ORDER BY` dentro de la CTE (salvo con `TOP`). El orden va en la consulta final.
- Se pueden poner nombres a las columnas: `WITH nombre (col1, col2) AS (...)`.

## 1.8. CTE encadenadas

Podemos tener varias CTE separadas por **coma** (solo un `WITH` al principio):

```
WITH facturas_t3 AS (  
    SELECT *  
    FROM facturas  
    WHERE fecha_emision >= '2026-07-01'  
    AND fecha_emision <= '2026-09-30'  
) ,  
enriquecidas AS (  
    SELECT  
        f.factura_id,  
        f.importe,  
        c.pais,  
        c.segmento  
    FROM facturas_t3 f  
    INNER JOIN clientes c  
        ON c.cliente_id = f.cliente_id  
) ,  
por_segmento AS (  
    SELECT  
        pais,  
        segmento,  
        COUNT(*) AS facturas,  
        SUM(importe) AS importe  
    FROM enriquecidas  
    GROUP BY pais, segmento  
)  
SELECT *  
FROM por_segmento;
```

La segunda CTE puede utilizar la primera y la tercera puede utilizar la segunda.

 Error típico: poner `WITH` otra vez antes de la segunda CTE. Solo hay **un** `WITH`; las demás se separan con `,`.

## 1.9. CTE recursiva

Se utiliza para recorrer estructuras jerárquicas.

Ejemplo:

```
WITH jerarquia AS (  
    -- Caso inicial  
    SELECT  
        unidad_id,  
        nombre,  
        unidad_padre_id,  
        0 AS nivel  
    FROM unidades  
    WHERE unidad_padre_id IS NULL  
  
    UNION ALL  
  
    -- Parte recursiva  
    SELECT  
        u.unidad_id,  
        u.nombre,  
        u.unidad_padre_id,  
        j.nivel + 1  
    FROM unidades u  
    INNER JOIN jerarquia j  
        ON u.unidad_padre_id = j.unidad_id  
)  
SELECT *  
FROM jerarquia;
```

La primera parte encuentra la raíz.

La segunda va buscando sus hijos.

```
Nivel 0  
  ↓  
Nivel 1  
  ↓  
Nivel 2  
  ↓  
Nivel 3
```

## Reglas de oro

1. Tiene **dos partes** unidas con **UNION ALL**: **ancla** (punto de partida) y **recursión** (se une a la propia CTE).
2. Las dos partes deben tener **las mismas columnas y los mismos tipos de datos**. Si construyes una ruta con texto, haz **CAST(... AS nvarchar(1000))** en las dos.
3. El **JOIN** de la parte recursiva es siempre: **tabla.<columna\_hijo→padre> = jerarquia.<id>**.
4. Por defecto para a las **100 iteraciones**; se cambia con **OPTION (MAXRECURSION n)** al final (0 = sin límite).

Esquema del JOIN recursivo:

```
hijo.padre_id = cte.id
└ el hijo apunta al padre que ya está en la CTE
```

---

## 1.10. Vistas

Una vista es una consulta guardada que se puede consultar como si fuera una tabla. **No guarda datos**: cada vez que la consultas, ejecuta la consulta sobre los datos actuales.

```
CREATE OR ALTER VIEW dbo.mi_vista
AS
SELECT
    cliente_id,
    razon_social
FROM clientes;
GO
```

Después:

```
SELECT *
FROM dbo.mi_vista;
```

### CREATE OR ALTER

Es muy útil en ejercicios porque permite ejecutar el script varias veces:

```
CREATE OR ALTER VIEW ...
```

### Reglas

- **CREATE VIEW** debe ser **la primera instrucción del lote** → pon **GO** antes y/o después.
- Dentro de la vista **no se puede usar ORDER BY** (salvo con **TOP**). Ordena al consultarla.
- Todas las columnas deben tener **nombre** (usa alias en expresiones como **SUM(...) AS total**).
- Se puede usar una CTE dentro de una vista: **CREATE VIEW ... AS WITH ... SELECT ....**
- Borrar: **DROP VIEW IF EXISTS dbo.mi\_vista;**

---

## 1.11. SELECT INTO

En SQL Server, **SELECT INTO** permite crear una tabla a partir del resultado de una consulta.

```
SELECT
    factura_id,
    cliente_id,
    importe
INTO dbo.facturas_copia
FROM facturas;
```

La tabla se crea automáticamente. Es una **copia estática**: si luego cambian los datos originales, la copia no cambia.

Si queremos poder ejecutar el script varias veces:

```
DROP TABLE IF EXISTS dbo.facturas_copia;

SELECT ...
INTO dbo.facturas_copia
FROM facturas;
```

## Reglas

- **INTO** va **entre el SELECT y el FROM**.
- La tabla destino **no puede existir** (si existe → error "There is already an object named..."). De ahí el **DROP TABLE IF EXISTS**.
- No copia claves primarias, índices ni restricciones; solo columnas y datos.
- Con CTE: el **WITH** va primero y el **INTO** dentro del **SELECT** final (ver Pregunta 11).
- Para crear una tabla vacía con la misma estructura: **SELECT ... INTO nueva FROM origen WHERE 1 = 0;**

---

## 1.12. Tablas temporales

Una tabla temporal comienza por #.

```
CREATE TABLE #ventas (
    cliente_id int,
    importe decimal(12,2)
);
```

Solo existe durante la sesión.

También podemos crearla con **SELECT INTO**:

```
SELECT
    cliente_id,
    SUM(importe) AS importe
```

```
INTO #ventas
FROM facturas
GROUP BY cliente_id;
```

Cuando termina la sesión, desaparece.

| Tipo              | Nombre    | Dura                                     | Quién la ve          |
|-------------------|-----------|------------------------------------------|----------------------|
| Temporal local    | #tabla    | hasta cerrar la sesión                   | solo tu sesión       |
| Temporal global   | ##tabla   | hasta cerrar la última sesión que la usa | todas las sesiones   |
| Variable de tabla | @tabla    | hasta fin del lote                       | solo el lote         |
| Tabla física      | dbo.tabla | permanente                               | todos (con permisos) |

Para poder repetir el script: `DROP TABLE IF EXISTS #ventas;`

Comparativa: ¿vista, SELECT INTO o temporal?

|                                      | Vista                                    | SELECT INTO<br>(tabla física) | #temporal                    |
|--------------------------------------|------------------------------------------|-------------------------------|------------------------------|
| ¿Guarda datos?                       | ✗ No (ejecuta la consulta cada vez)      | ✓ Sí                          | ✓ Sí                         |
| ¿Se actualiza sola con datos nuevos? | ✓ Sí                                     | ✗ No (foto estática)          | ✗ No                         |
| ¿Sobrevive a cerrar sesión?          | ✓ Sí (es un objeto)                      | ✓ Sí                          | ✗ No                         |
| ¿La ven otros usuarios?              | ✓ Sí                                     | ✓ Sí                          | ✗ No                         |
| Úsala para...                        | exponer una consulta siempre actualizada | congelar una foto/copia       | pasos intermedios de trabajo |

## 1.13. COALESCE

Sirve para sustituir `NULL` por otro valor. Devuelve el **primer valor no nulo** de la lista.

```
COALESCE(cobrado, 0)
```

Si `cobrado` es `NULL`, devuelve `0`.

Ejemplo:

```
SELECT
    cliente_id,
```

```
COALESCE(SUM(importe), 0) AS importe
FROM cobros
GROUP BY cliente_id;
```

Con varios valores: `COALESCE(telefono_movil, telefono_fijo, 'Sin teléfono')`.

---

## 1.14. CASE

Permite crear condiciones.

```
CASE
  WHEN pendiente = 0 THEN 'Cobrada'
  WHEN cobrado = 0 THEN 'Sin cobro'
  ELSE 'Cobro parcial'
END
```

Se utiliza para transformar valores según una condición.

### Reglas

- Se evalúa **de arriba abajo** y gana **el primer WHEN verdadero**. El orden importa.
- Si ninguno se cumple y no hay `ELSE`, devuelve `NULL`.
- Termina siempre en `END` y suele llevar alias: `END AS estado`.

### Dos formas

```
-- Forma "buscada" (condiciones libres)
CASE WHEN importe > 1000 THEN 'Alta' WHEN importe > 100 THEN 'Media' ELSE 'Baja'
END

-- Forma "simple" (compara un valor con igualdad)
CASE pais WHEN 'ES' THEN 'España' WHEN 'FR' THEN 'Francia' ELSE 'Otro' END
```

### CASE dentro de un agregado (agregación condicional)

Muy frecuente en exámenes (ver Pregunta 9):

```
SELECT
  cliente_id,
  SUM(CASE WHEN pendiente > 0 THEN pendiente ELSE 0 END) AS saldo,
  COUNT(CASE WHEN pais = 'ES' THEN 1 END) AS facturas_es --
COUNT ignora NULL
FROM ...
GROUP BY cliente_id;
```

---

## 1.15. Fechas

Para el T3 de 2026:

```
WHERE fecha >= '2026-07-01'
AND fecha <= '2026-09-30'
```

También se puede utilizar:

```
WHERE fecha BETWEEN '2026-07-01' AND '2026-09-30'
```

En esta práctica:

Fecha de corte → 30/09/2026

T3 → 01/07/2026 - 30/09/2026

**Nota de seguridad:** si la columna es `date`, `<= '2026-09-30'` es correcto. Si fuera `datetime/datetime2` (con hora), una factura del 30/09 a las 10:00 **quedaría fuera**. La forma segura para ambos casos es:

```
WHERE fecha >= '2026-07-01'
AND fecha < '2026-10-01'      -- "menor que el día siguiente"
```

Formato recomendado de literales de fecha: `'YYYY-MM-DD'` (o `'YYYYMMDD'`), que SQL Server interpreta igual con cualquier idioma.

Trimestres de 2026:

| Trimestre | Desde      | Hasta      |
|-----------|------------|------------|
| T1        | 2026-01-01 | 2026-03-31 |
| T2        | 2026-04-01 | 2026-06-30 |
| T3        | 2026-07-01 | 2026-09-30 |
| T4        | 2026-10-01 | 2026-12-31 |

---

## 1.16. GROUP BY y HAVING

`GROUP BY` agrupa filas para poder calcular agregados (`SUM`, `COUNT`...) por grupo.

```
SELECT
    pais,
    COUNT(*)      AS facturas,
    SUM(importe) AS total
FROM facturas f
JOIN clientes c ON c.cliente_id = f.cliente_id
WHERE f.fecha_emision >= '2026-07-01'      -- filtra FILAS (antes de agrupar)
GROUP BY pais
HAVING SUM(importe) >= 200000              -- filtra GRUPOS (después de agrupar)
ORDER BY total DESC;
```

**Regla de oro:** toda columna del **SELECT** que **no** esté dentro de una función de agregado **debe estar en el GROUP BY**. Si no: *"Column ... is invalid in the select list because it is not contained in either an aggregate function or the GROUP BY clause"*.

|                         | WHERE              | HAVING               |
|-------------------------|--------------------|----------------------|
| Filtra                  | filas individuales | grupos ya calculados |
| Se ejecuta              | antes de agrupar   | después de agrupar   |
| ¿Admite SUM(), COUNT()? | ❌ No               | ✅ Sí                 |

## 2. Diccionario de funciones

Consulta rápida: **qué hace** → **sintaxis** → **ejemplo** → **resultado**. Los ejemplos con **SELECT** sin **FROM** se pueden probar tal cual.

### 2.1. Funciones de agregación (COUNT, SUM, AVG, MIN, MAX...)

Resumen de muchas filas en un único valor (o uno por grupo con **GROUP BY**).

| Función                 | Qué hace                                                      | Ejemplo                                          |
|-------------------------|---------------------------------------------------------------|--------------------------------------------------|
| COUNT(*)                | Cuenta <b>todas las filas</b> (incluidas las que tienen NULL) | SELECT COUNT(*) FROM facturas;                   |
| COUNT(columna)          | Cuenta filas donde la columna <b>NO es NULL</b>               | SELECT COUNT(fecha_vencimiento) FROM facturas;   |
| COUNT(DISTINCT columna) | Cuenta valores <b>distintos</b> no nulos                      | SELECT COUNT(DISTINCT cliente_id) FROM facturas; |
| SUM(columna)            | Suma (ignora NULL)                                            | SELECT SUM(importe) FROM facturas;               |
| AVG(columna)            | Media (ignora NULL)                                           | SELECT AVG(importe) FROM facturas;               |
| MIN(columna)            | Mínimo (sirve para números, texto y fechas)                   | SELECT MIN(fecha_emision) FROM facturas;         |

| Función                            | Qué hace                                                       | Ejemplo                                           |
|------------------------------------|----------------------------------------------------------------|---------------------------------------------------|
| <code>MAX(columna)</code>          | Máximo                                                         | <code>SELECT MAX(importe) FROM facturas;</code>   |
| <code>STRING_AGG(col, sep)</code>  | <b>Concatena</b> los valores de un grupo en un texto           | ver abajo                                         |
| <code>STDEV(col) / VAR(col)</code> | Desviación típica / varianza (muestral)                        | <code>SELECT STDEV(importe) FROM facturas;</code> |
| <code>COUNT_BIG(*)</code>          | Igual que <code>COUNT</code> pero devuelve <code>bigint</code> | para tablas enormes                               |

```
-- Varias agregaciones a la vez
SELECT
    COUNT(*)                AS num_facturas,
    COUNT(DISTINCT cliente_id) AS clientes_distintos,
    SUM(importe)             AS total,
    AVG(importe)             AS media,
    MIN(importe)             AS minimo,
    MAX(importe)             AS maximo
FROM facturas;
```

```
-- Por grupo
SELECT cliente_id, COUNT(*) AS facturas, SUM(importe) AS total
FROM facturas
GROUP BY cliente_id;
```

```
-- STRING_AGG: lista de facturas por cliente en una sola celda
SELECT
    cliente_id,
    STRING_AGG(CAST(factura_id AS varchar(20)), ', ')
        WITHIN GROUP (ORDER BY factura_id) AS facturas
FROM facturas
GROUP BY cliente_id;
-- Resultado: 7 | 101, 105, 230
```

## Diferencias que suelen preguntar

| Expresión                        | ¿Cuenta NULL? | Ejemplo con valores 10, NULL, 10, 20 |
|----------------------------------|---------------|--------------------------------------|
| <code>COUNT(*)</code>            | Sí            | 4                                    |
| <code>COUNT(col)</code>          | No            | 3                                    |
| <code>COUNT(DISTINCT col)</code> | No            | 2                                    |

⚠ **AVG de enteros trunca:** `AVG(cantidad)` con 1, 2 da 1, no 1.5. Solución: `AVG(CAST(cantidad AS decimal(12,2)))` o `AVG(cantidad * 1.0)`.

⚠ **SUM sobre un LEFT JOIN sin coincidencias da NULL**, no 0 → `COALESCE(SUM(x), 0)`.

⚠ **COUNT(\*) tras un LEFT JOIN cuenta 1 aunque no haya coincidencia** (la fila con NULLs existe). Para contar de verdad hijos: `COUNT(f.factura_id)`.

## 2.2. Funciones de texto (**CONCAT**, **LEN**, **SUBSTRING**...)

| Función                                      | Qué hace                                            | Ejemplo                                             | Resultado   |
|----------------------------------------------|-----------------------------------------------------|-----------------------------------------------------|-------------|
| <code>CONCAT(a, b, ...)</code>               | Une textos; <b>trata NULL como vacío</b>            | <code>CONCAT('Ana', ' ', 'Pérez')</code>            | Ana Pérez   |
| <code>CONCAT_WS(sep, a, b, ...)</code>       | Une con separador; <b>omite los NULL</b>            | <code>CONCAT_WS(' - ', 'ES', NULL, 'Madrid')</code> | ES - Madrid |
| <code>+</code>                               | Une textos; <b>si hay NULL el resultado es NULL</b> | <code>'Ana' + ' ' + 'Pérez'</code>                  | Ana Pérez   |
| <code>LEN(s)</code>                          | Nº de caracteres (ignora espacios finales)          | <code>LEN('Hola')</code>                            | 4           |
| <code>DATALength(s)</code>                   | Nº de <b>bytes</b>                                  | <code>DATALength('Hola')</code>                     | 4           |
| <code>UPPER(s) / LOWER(s)</code>             | Mayúsculas / minúsculas                             | <code>UPPER('es')</code>                            | ES          |
| <code>LEFT(s, n)</code>                      | Primeros n caracteres                               | <code>LEFT('Barcelona', 3)</code>                   | Bar         |
| <code>RIGHT(s, n)</code>                     | Últimos n caracteres                                | <code>RIGHT('Barcelona', 3)</code>                  | ona         |
| <code>SUBSTRING(s, inicio, long)</code>      | Extrae trozo ( <b>empieza en 1</b> )                | <code>SUBSTRING('Barcelona', 2, 4)</code>           | arce        |
| <code>CHARINDEX(buscado, s [, desde])</code> | Posición de un texto (0 si no está)                 | <code>CHARINDEX('@', 'ana@mail.com')</code>         | 4           |
| <code>PATINDEX('%patrón%', s)</code>         | Posición de un patrón con comodines                 | <code>PATINDEX('%[0-9]%', 'ab3c')</code>            | 3           |
| <code>REPLACE(s, viejo, nuevo)</code>        | Sustituye todas las apariciones                     | <code>REPLACE('a-b-c', '-', '/')</code>             | a/b/c       |
| <code>TRIM(s)</code>                         | Quita espacios a ambos lados                        | <code>TRIM(' hola ')</code>                         | hola        |
| <code>LTRIM(s) / RTRIM(s)</code>             | Quita espacios a izquierda / derecha                | <code>LTRIM(' hola')</code>                         | hola        |

| Función                                | Qué hace                                            | Ejemplo                                      | Resultado  |
|----------------------------------------|-----------------------------------------------------|----------------------------------------------|------------|
| STUFF(s, inicio, long, nuevo)          | Borra <b>long</b> caracteres e inserta <b>nuevo</b> | STUFF('abcdef', 2, 3, 'XY')                  | aXYef      |
| REVERSE(s)                             | Invierte el texto                                   | REVERSE('abc')                               | cba        |
| REPLICATE(s, n)                        | Repite un texto n veces                             | REPLICATE('ab', 3)                           | ababab     |
| SPACE(n)                               | n espacios                                          | 'A' + SPACE(2) + 'B'                         | A B        |
| FORMAT(valor, 'formato' [, 'cultura']) | Da formato a números/fechas (devuelve texto)        | FORMAT(1234.5, 'N2', 'es-ES')                | 1.234,50   |
| STRING_SPLIT(s, sep)                   | Parte un texto en filas (columna <b>value</b> )     | SELECT value FROM STRING_SPLIT('a,b,c', ',') | 3 filas    |
| TRANSLATE(s, orig, dest)               | Sustituye carácter a carácter                       | TRANSLATE('2*3', '*', 'x')                   | 2x3        |
| QUOTENAME(s)                           | Pone corchetes (para nombres de objeto)             | QUOTENAME('mi tabla')                        | [mi tabla] |
| ASCII(c) / CHAR(n)                     | Código ↔ carácter                                   | ASCII('A')                                   | 65         |
| UNICODE(c) / NCHAR(n)                  | Igual para Unicode                                  | NCHAR(65)                                    | A          |

```
-- Nombre completo
SELECT CONCAT(nombre, ' ', apellidos) AS nombre_completo FROM empleados;

-- Iniciales
SELECT CONCAT(LEFT(nombre, 1), LEFT(apellidos, 1)) AS iniciales FROM empleados;

-- Dominio de un email
SELECT SUBSTRING(email, CHARINDEX('@', email) + 1, LEN(email)) AS dominio FROM clientes;

-- Código con ceros a la izquierda: 7 → 00007
SELECT RIGHT(CONCAT('00000', 7), 5);

-- Buscar texto
SELECT * FROM clientes WHERE razon_social LIKE '%Iberia%';
```

## CONCAT vs +

```
SELECT 'Hola ' + NULL;           -- NULL ← el + propaga el NULL
SELECT CONCAT('Hola ', NULL);    -- 'Hola ' ← CONCAT lo ignora
```

Con números, + suma en lugar de concatenar: 'Factura ' + 5 da error → usa `CONCAT('Factura ', 5)` o `CAST(5 AS varchar)`.

## LIKE y comodines

| Patrón | Significado                      | Ejemplo                          |
|--------|----------------------------------|----------------------------------|
| %      | cualquier cantidad de caracteres | LIKE 'Ib%' empieza por Ib        |
| _      | exactamente un carácter          | LIKE '_a%' segunda letra a       |
| [abc]  | uno de estos caracteres          | LIKE '[AB]%' empieza por A o B   |
| [a-f]  | rango                            | LIKE '[0-9]%' empieza por dígito |
| [^abc] | cualquiera menos estos           | LIKE '[^0-9]%'                   |

```
WHERE razon_social LIKE '%S.L.'      -- termina en S.L.
WHERE razon_social NOT LIKE '%S.A.%' -- no contiene S.A.
```

## 2.3. Funciones de fecha y hora

| Función                 | Qué hace                                | Ejemplo                         | Resultado                  |
|-------------------------|-----------------------------------------|---------------------------------|----------------------------|
| GETDATE()               | Fecha y hora actual<br>(datetime)       | SELECT GETDATE();               | 2026-10-08<br>10:30:00.000 |
| SYSDATETIME()           | Igual, más precisión<br>(datetime2)     | SELECT SYSDATETIME();           |                            |
| CURRENT_TIMESTAMP       | Sinónimo ANSI de<br>GETDATE()           |                                 |                            |
| GETUTCDATE()            | Fecha/hora UTC                          |                                 |                            |
| CAST(GETDATE() AS date) | Solo la fecha de hoy                    |                                 | 2026-10-08                 |
| YEAR(f)                 | Año                                     | YEAR('2026-09-30')              | 2026                       |
| MONTH(f)                | Mes                                     | MONTH('2026-09-30')             | 9                          |
| DAY(f)                  | Día                                     | DAY('2026-09-30')               | 30                         |
| DATEPART(parte, f)      | Parte de la fecha como<br><b>número</b> | DATEPART(QUARTER, '2026-09-30') | 3                          |
| DATENAME(parte, f)      | Parte como <b>texto</b>                 | DATENAME(MONTH, '2026-09-30')   | September*                 |
| DATEADD(parte, n, f)    | Suma/resta n unidades                   | DATEADD(DAY, 30, '2026-09-01')  | 2026-10-01                 |

| Función                   | Qué hace                                                | Ejemplo                                   | Resultado  |
|---------------------------|---------------------------------------------------------|-------------------------------------------|------------|
| DATEDIFF(parte, ini, fin) | Nº de <b>límites</b> de parte cruzados entre dos fechas | DATEDIFF(DAY, '2026-09-01', '2026-09-30') | 29         |
| EOMONTH(f [, n])          | Último día del mes (n meses después)                    | EOMONTH('2026-09-10')                     | 2026-09-30 |
| DATEFROMPARTS(y, m, d)    | Construye una fecha                                     | DATEFROMPARTS(2026, 9, 30)                | 2026-09-30 |
| DATETRUNC(parte, f)       | Trunca al inicio de la parte (SQL Server 2022+)         | DATETRUNC(QUARTER, '2026-08-15')          | 2026-07-01 |
| ISDATE(texto)             | ¿Es una fecha válida? (1/0)                             | ISDATE('2026-02-30')                      | 0          |
| FORMAT(f, 'dd/MM/yyyy')   | Fecha como texto con formato                            | FORMAT(GETDATE(), 'dd/MM/yyyy')           | 08/10/2026 |

\* DATENAME depende del idioma de la sesión (SET LANGUAGE Spanish; para nombres en español).

### Partes de fecha más usadas (parte)

| Parte     | Abreviatura | Parte   | Abreviatura |
|-----------|-------------|---------|-------------|
| YEAR      | yy, yyyy    | WEEK    | wk, ww      |
| QUARTER   | qq, q       | WEEKDAY | dw          |
| MONTH     | mm, m       | HOURL   | hh          |
| DAY       | dd, d       | MINUTE  | mi, n       |
| DAYOFYEAR | dy, y       | SECOND  | ss, s       |

```
-- Facturas del T3 de 2026 usando funciones
WHERE YEAR(fecha_emision) = 2026
AND DATEPART(QUARTER, fecha_emision) = 3
-- ⚠️ Correcto pero MENOS eficiente (impide usar índices). En el examen está bien;
-- en la práctica es preferible el rango de fechas.

-- Días de retraso en el cobro
SELECT factura_id,
       DATEDIFF(DAY, fecha_vencimiento, '2026-09-30') AS dias_vencida
FROM facturas
WHERE fecha_vencimiento < '2026-09-30';

-- Primer y último día del mes de una fecha
SELECT DATEFROMPARTS(YEAR(f), MONTH(f), 1) AS inicio_mes,
```

```

        EOMONTH(f)                                AS fin_mes
FROM (SELECT CAST('2026-09-15' AS date) AS f) x;

-- Vencimiento a 30 días
SELECT DATEADD(DAY, 30, fecha_emision) AS vence FROM facturas;

-- Agrupar por mes
SELECT YEAR(fecha_emision) AS anio, MONTH(fecha_emision) AS mes, SUM(importe) AS
total
FROM facturas
GROUP BY YEAR(fecha_emision), MONTH(fecha_emision)
ORDER BY anio, mes;

```

⚠ **DATEDIFF** cuenta **fronteras cruzadas**, no tiempo transcurrido: `DATEDIFF(YEAR, '2026-12-31', '2027-01-01')` devuelve **1** aunque solo pasó un día.

## 2.4. Funciones numéricas

| Función                                 | Qué hace                              | Ejemplo                         | Resultado  |
|-----------------------------------------|---------------------------------------|---------------------------------|------------|
| <code>ROUND(n, dec)</code>              | Redondea a <code>dec</code> decimales | <code>ROUND(123.4567, 2)</code> | 123.4600   |
| <code>ROUND(n, -dec)</code>             | Redondea a decenas/centenas...        | <code>ROUND(1234, -2)</code>    | 1200       |
| <code>CEILING(n)</code>                 | Redondeo hacia arriba                 | <code>CEILING(4.1)</code>       | 5          |
| <code>FLOOR(n)</code>                   | Redondeo hacia abajo                  | <code>FLOOR(4.9)</code>         | 4          |
| <code>ABS(n)</code>                     | Valor absoluto                        | <code>ABS(-7)</code>            | 7          |
| <code>SIGN(n)</code>                    | Signo (-1, 0, 1)                      | <code>SIGN(-3)</code>           | -1         |
| <code>POWER(b, e)</code>                | Potencia                              | <code>POWER(2, 10)</code>       | 1024       |
| <code>SQRT(n)</code>                    | Raíz cuadrada                         | <code>SQRT(81)</code>           | 9          |
| <code>EXP(n) / LOG(n) / LOG10(n)</code> | Exponencial / logaritmos              | <code>LOG10(1000)</code>        | 3          |
| <code>PI()</code>                       | $\pi$                                 | <code>PI()</code>               | 3.14159... |
| <code>RAND()</code>                     | Aleatorio entre 0 y 1                 |                                 |            |
| <code>a % b</code>                      | Resto (módulo)                        | <code>10 % 3</code>             | 1          |

```

-- Porcentaje cobrado (cuidado con la división entera y por cero)
SELECT
    factura_id,
    ROUND(100.0 * cobrado / NULLIF(importe, 0), 2) AS pct_cobrado
FROM facturas_enriquecidas;

```

⚠ **División entera:** `SELECT 1/2` da 0. Usa `1.0/2` o `CAST(1 AS decimal)/2`.

⚠ **División por cero:** protege el divisor con `NULLIF(divisor, 0)` (devuelve NULL en vez de error).

## 2.5. Funciones para NULL y condicionales

| Función                           | Qué hace                                                                     | Ejemplo                                            | Resultado |
|-----------------------------------|------------------------------------------------------------------------------|----------------------------------------------------|-----------|
| <code>COALESCE(a, b, c...)</code> | Primer valor <b>no nulo</b> (estándar, admite N valores)                     | <code>COALESCE(NULL, NULL, 5)</code>               | 5         |
| <code>ISNULL(a, b)</code>         | Si <b>a</b> es NULL devuelve <b>b</b> (solo 2 valores, propio de SQL Server) | <code>ISNULL(NULL, 0)</code>                       | 0         |
| <code>NULLIF(a, b)</code>         | Devuelve NULL si <b>a = b</b> ; si no, <b>a</b>                              | <code>NULLIF(0, 0)</code>                          | NULL      |
| <code>IIF(cond, si, no)</code>    | Un <b>CASE</b> abreviado                                                     | <code>IIF(importe &gt; 100, 'Alto', 'Bajo')</code> |           |
| <code>CHOOSE(i, v1, v2...)</code> | Elige por posición (empieza en 1)                                            | <code>CHOOSE(2, 'a', 'b', 'c')</code>              | b         |
| <code>CASE WHEN ... END</code>    | Condicional completo                                                         | ver 1.14                                           |           |

Comparar con NULL:

```
WHERE columna IS NULL           -- ✓
WHERE columna IS NOT NULL       -- ✓
WHERE columna = NULL            -- ✗ nunca devuelve nada
WHERE columna <> 'x'             -- ⚠ también excluye los NULL
WHERE columna <> 'x' OR columna IS NULL -- para incluirlos
```

**COALESCE** vs **ISNULL**: ambos sirven para sustituir NULL. En el examen usa **COALESCE** (es estándar y admite más valores).

## 2.6. Funciones de conversión de tipo

| Función                  | Sintaxis                                     | Ejemplo                                           | Resultado  |
|--------------------------|----------------------------------------------|---------------------------------------------------|------------|
| <code>CAST</code>        | <code>CAST(valor AS tipo)</code>             | <code>CAST('2026-09-30' AS date)</code>           | 2026-09-30 |
| <code>CONVERT</code>     | <code>CONVERT(tipo, valor [, estilo])</code> | <code>CONVERT(varchar(10), GETDATE(), 103)</code> | 08/10/2026 |
| <code>TRY_CAST</code>    | Como CAST pero devuelve NULL si falla        | <code>TRY_CAST('abc' AS int)</code>               | NULL       |
| <code>TRY_CONVERT</code> | Como CONVERT pero devuelve NULL si falla     | <code>TRY_CONVERT(int, 'abc')</code>              | NULL       |

| Función              | Sintaxis                    | Ejemplo                                          | Resultado  |
|----------------------|-----------------------------|--------------------------------------------------|------------|
| PARSE /<br>TRY_PARSE | Convierte texto con cultura | TRY_PARSE('30/09/2026' AS date<br>USING 'es-ES') | 2026-09-30 |

Estilos de **CONVERT** para fechas más usados:

| Estilo | Formato             | Ejemplo             |
|--------|---------------------|---------------------|
| 23     | yyyy-mm-dd          | 2026-09-30          |
| 103    | dd/mm/yyyy          | 30/09/2026          |
| 120    | yyyy-mm-dd hh:mi:ss | 2026-09-30 14:05:00 |
| 112    | yyyymmdd            | 20260930            |

Tipos de datos más frecuentes:

| Tipo                              | Uso                                                                                           |
|-----------------------------------|-----------------------------------------------------------------------------------------------|
| int, bigint, smallint,<br>tinyint | enteros                                                                                       |
| decimal(p,s) /<br>numeric(p,s)    | importes exactos (p = dígitos totales, s = decimales). decimal(12,2) → hasta 9.999.999.999,99 |
| float, real                       | decimales aproximados (evitar para dinero)                                                    |
| bit                               | verdadero/falso (0/1)                                                                         |
| varchar(n) / nvarchar(n)          | texto (la n admite Unicode). varchar(max) para textos largos                                  |
| char(n) / nchar(n)                | texto de longitud fija                                                                        |
| date                              | solo fecha                                                                                    |
| datetime2 / datetime              | fecha y hora                                                                                  |
| uniqueidentifier                  | GUID                                                                                          |

## 2.7. Funciones de ventana (**OVER**)

Calculan algo **sobre un conjunto de filas relacionado con la fila actual, sin colapsar filas** (a diferencia de **GROUP BY**).

```
función() OVER ( PARTITION BY <grupos> ORDER BY <orden> [marco] )
                |   ^ reinicia el cálculo en cada grupo (opcional)
                |   ^ define el orden dentro del grupo
```

| Función                                            | Qué hace                                                                  |
|----------------------------------------------------|---------------------------------------------------------------------------|
| <code>ROW_NUMBER()</code>                          | Numera 1, 2, 3... sin repetir (desempata arbitrariamente)                 |
| <code>RANK()</code>                                | Ranking con empates; <b>salta</b> números (1, 1, 3)                       |
| <code>DENSE_RANK()</code>                          | Ranking con empates; <b>no salta</b> (1, 1, 2)                            |
| <code>NTILE(n)</code>                              | Reparte en n grupos del mismo tamaño (cuartiles → <code>NTILE(4)</code> ) |
| <code>LAG(col, n, def)</code>                      | Valor de la fila <b>anterior</b> (n filas atrás)                          |
| <code>LEAD(col, n, def)</code>                     | Valor de la fila <b>siguiente</b>                                         |
| <code>FIRST_VALUE(col) / LAST_VALUE(col)</code>    | Primer / último valor de la ventana                                       |
| <code>SUM/AVG/COUNT/MIN/MAX(col) OVER (...)</code> | Agregados "acumulados" o por grupo sin colapsar                           |

```
-- Ranking de facturas por importe dentro de cada cliente
SELECT
    cliente_id, factura_id, importe,
    ROW_NUMBER() OVER (PARTITION BY cliente_id ORDER BY importe DESC) AS pos
FROM facturas;

-- Mayor factura de cada cliente (top 1 por grupo) → hay que envolverlo en CTE
WITH ranking AS (
    SELECT cliente_id, factura_id, importe,
           ROW_NUMBER() OVER (PARTITION BY cliente_id ORDER BY importe DESC) AS
pos
    FROM facturas
)
SELECT cliente_id, factura_id, importe
FROM ranking
WHERE pos = 1;
-- ⚠ No se puede poner ROW_NUMBER() directamente en el WHERE

-- Total acumulado por fecha
SELECT
    fecha_emision, importe,
    SUM(importe) OVER (ORDER BY fecha_emision, factura_id
                      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS
acumulado
FROM facturas;

-- Importe de cada factura vs la media (alternativa a la subconsulta escalar)
SELECT factura_id, importe, AVG(importe) OVER () AS media_global
FROM facturas;

-- Variación respecto a la factura anterior del mismo cliente
SELECT cliente_id, fecha_emision, importe,
       importe - LAG(importe) OVER (PARTITION BY cliente_id ORDER BY
```

```
fecha_emision) AS variacion
FROM facturas;
```

---

## 2.8. Funciones de agrupación avanzada

```
-- ROLLUP: subtotales jerárquicos + total general
SELECT pais, segmento, SUM(importe) AS total
FROM enriquecidas
GROUP BY ROLLUP (pais, segmento);

-- CUBE: todas las combinaciones de subtotales
GROUP BY CUBE (pais, segmento)

-- GROUPING SETS: combinaciones a medida
GROUP BY GROUPING SETS ((pais), (segmento), ())

-- GROUPING(col) devuelve 1 si la fila es un subtotal de esa columna (para
etiquetar "TOTAL")
SELECT CASE WHEN GROUPING(pais) = 1 THEN 'TOTAL' ELSE pais END AS pais,
        SUM(importe) AS total
FROM enriquecidas
GROUP BY ROLLUP (pais);
```

---

## 3. Otros comandos útiles

### 3.1. DISTINCT, TOP, OFFSET-FETCH

```
-- Valores únicos
SELECT DISTINCT pais FROM clientes;
SELECT DISTINCT pais, segmento FROM clientes;      -- combinaciones únicas

-- Las N primeras filas (SIEMPRE con ORDER BY para que "primeras" tenga sentido)
SELECT TOP 3 *
FROM facturas
ORDER BY importe DESC;

-- Incluir empates en la última posición
SELECT TOP 3 WITH TIES *
FROM facturas
ORDER BY importe DESC;

-- Porcentaje de filas
SELECT TOP 10 PERCENT * FROM facturas ORDER BY importe DESC;

-- Paginación: saltar 20 filas y coger 10 (requiere ORDER BY)
```

```
SELECT *  
FROM facturas  
ORDER BY factura_id  
OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY;
```

## 3.2. Operadores de conjuntos

Combinan **resultados de dos consultas** una debajo de otra. Ambas deben tener **el mismo número de columnas y tipos compatibles**; los nombres de columna los toma de la **primera**.

| Operador  | Qué hace                                                                 |
|-----------|--------------------------------------------------------------------------|
| UNION ALL | Junta todas las filas (con duplicados). Más rápido. <b>El más usado.</b> |
| UNION     | Junta y <b>elimina duplicados</b>                                        |
| INTERSECT | Solo filas que están en <b>ambas</b>                                     |
| EXCEPT    | Filas de la primera que <b>no</b> están en la segunda                    |

```
SELECT cliente_id FROM facturas  
UNION ALL  
SELECT cliente_id FROM cobros;  
  
-- Clientes con facturas pero sin ningún cobro  
SELECT cliente_id FROM facturas  
EXCEPT  
SELECT f.cliente_id FROM cobros co JOIN facturas f ON f.factura_id =  
co.factura_id;  
  
-- El ORDER BY va solo al final de todo  
SELECT 'A' AS origen, importe FROM facturas  
UNION ALL  
SELECT 'B', importe FROM gastos  
ORDER BY importe DESC;
```

## 3.3. Operadores de comparación y lógicos

| Operador                   | Significado                                                   |
|----------------------------|---------------------------------------------------------------|
| =, <> (o !=), <, <=, >, >= | comparaciones                                                 |
| BETWEEN a AND b            | entre a y b, <b>ambos incluidos</b>                           |
| IN (...) / NOT IN (...)    | pertenece / no pertenece a una lista                          |
| LIKE                       | patrón de texto                                               |
| IS NULL / IS NOT NULL      | nulidad                                                       |
| AND, OR, NOT               | lógicos ( <b>AND se evalúa antes que OR</b> → usa paréntesis) |

| Operador            | Significado                                                  |
|---------------------|--------------------------------------------------------------|
| EXISTS / NOT EXISTS | existencia de filas en subconsulta                           |
| ANY / SOME / ALL    | comparar con cualquiera/todos los valores de una subconsulta |

```
WHERE (pais = 'ES' OR pais = 'FR') AND importe > 1000 -- con paréntesis
WHERE importe > ALL (SELECT importe FROM gastos)      -- mayor que todos
```

### 3.4. INSERT, UPDATE, DELETE, TRUNCATE

```
-- INSERT una fila
INSERT INTO clientes (cliente_id, razon_social, pais)
VALUES (500, 'Nueva SL', 'ES');

-- INSERT varias filas
INSERT INTO clientes (cliente_id, razon_social, pais)
VALUES (501, 'Alfa SA', 'FR'),
       (502, 'Beta SL', 'PT');

-- INSERT desde un SELECT (la tabla destino YA existe)
INSERT INTO dbo.cobros (cobro_id, factura_id, fecha_cobro, importe, metodo)
SELECT cobro_id, factura_id, fecha_cobro, importe, metodo
FROM #lote_valido;

-- UPDATE (¡siempre con WHERE o cambias todas las filas!)
UPDATE facturas
SET importe = importe * 1.21
WHERE factura_id = 101;

-- UPDATE con JOIN
UPDATE f
SET f.estado = 'Vencida'
FROM facturas f
JOIN clientes c ON c.cliente_id = f.cliente_id
WHERE f.fecha_vencimiento < '2026-09-30' AND c.pais = 'ES';

-- DELETE (¡siempre con WHERE!)
DELETE FROM cobros WHERE importe <= 0;

-- TRUNCATE: vacía toda la tabla (rápido, sin WHERE, reinicia IDENTITY)
TRUNCATE TABLE dbo.stg_cobros;
```

|              | DELETE            | TRUNCATE        | DROP TABLE                                 |
|--------------|-------------------|-----------------|--------------------------------------------|
| Borra        | filas (con WHERE) | todas las filas | <b>toda la tabla</b> (estructura incluida) |
| Admite WHERE | ✓                 | ✗               | ✗                                          |

|                   | DELETE | TRUNCATE | DROP TABLE |
|-------------------|--------|----------|------------|
| Reinicia IDENTITY | ✗      | ✓        | —          |

Truco antes de UPDATE/DELETE: escribe primero el SELECT con el mismo WHERE y comprueba qué filas afectaría.

### 3.5. MERGE (insertar o actualizar)

```
MERGE dbo.clientes AS destino
USING dbo.stg_clientes AS origen
  ON destino.cliente_id = origen.cliente_id
WHEN MATCHED THEN
  UPDATE SET destino.razon_social = origen.razon_social
WHEN NOT MATCHED BY TARGET THEN
  INSERT (cliente_id, razon_social) VALUES (origen.cliente_id,
origen.razon_social);
```

### 3.6. CREATE TABLE y restricciones

```
CREATE TABLE dbo.clientes (
  cliente_id      int          NOT NULL PRIMARY KEY,
  razon_social    nvarchar(200) NOT NULL,
  pais            char(2)       NOT NULL DEFAULT 'ES',
  limite_credito  decimal(12,2) NULL,
  fecha_alta      date          NOT NULL DEFAULT GETDATE(),
  email           varchar(150)  UNIQUE,
  CONSTRAINT ck_limite CHECK (limite_credito >= 0)
);

CREATE TABLE dbo.facturas (
  factura_id      int IDENTITY(1,1) PRIMARY KEY, -- autonumérico
  cliente_id      int NOT NULL,
  importe         decimal(12,2) NOT NULL,
  CONSTRAINT fk_fact_cli FOREIGN KEY (cliente_id) REFERENCES dbo.clientes
(cliente_id)
);
```

| Restricción                | Para qué sirve                                |
|----------------------------|-----------------------------------------------|
| PRIMARY KEY                | identifica cada fila (único + no nulo)        |
| FOREIGN KEY ... REFERENCES | obliga a que el valor exista en la otra tabla |
| UNIQUE                     | no permite repetidos                          |
| NOT NULL                   | no permite NULL                               |
| DEFAULT                    | valor por defecto si no se indica             |

| Restricción                  | Para qué sirve               |
|------------------------------|------------------------------|
| CHECK                        | condición que debe cumplirse |
| IDENTITY(inicio, incremento) | numeración automática        |

### 3.7. ALTER y DROP

```

ALTER TABLE dbo.clientes ADD telefono varchar(20) NULL;           -- añadir columna
ALTER TABLE dbo.clientes ALTER COLUMN telefono varchar(30);      -- cambiar tipo
ALTER TABLE dbo.clientes DROP COLUMN telefono;                   -- borrar columna
ALTER TABLE dbo.clientes ADD CONSTRAINT ck_x CHECK (limite_credito >= 0);

DROP TABLE IF EXISTS dbo.facturas_copia;
DROP VIEW IF EXISTS dbo.mi_vista;

CREATE INDEX ix_facturas_cliente ON dbo.facturas (cliente_id);

EXEC sp_rename 'dbo.tabla_vieja', 'tabla_nueva';                  -- renombrar

```

### 3.8. Variables, IF y WHILE

```

DECLARE @corte date = '2026-09-30';
DECLARE @pais char(2);
SET @pais = 'ES';

SELECT * FROM facturas
WHERE fecha_emision <= @corte;

-- Guardar el resultado de una consulta en una variable
DECLARE @media decimal(12,2);
SELECT @media = AVG(importe) FROM facturas;

IF @media > 1000
    PRINT 'Media alta';
ELSE
    PRINT 'Media baja';

-- Varios comandos → BEGIN ... END
IF EXISTS (SELECT 1 FROM clientes WHERE pais = 'XX')
BEGIN
    PRINT 'Hay clientes con país XX';
    DELETE FROM clientes WHERE pais = 'XX';
END

-- Bucle
DECLARE @i int = 1;
WHILE @i <= 5
BEGIN
    PRINT @i;

```

```
SET @i += 1;  
END
```

### 3.9. Transacciones y control de errores

```
BEGIN TRY  
    BEGIN TRANSACTION;  
  
    INSERT INTO dbo.cobros (...) SELECT ... FROM #lote_valido;  
    UPDATE dbo.facturas SET ... WHERE ...;  
  
    COMMIT TRANSACTION;           -- confirma todo  
END TRY  
BEGIN CATCH  
    ROLLBACK TRANSACTION;         -- deshace todo si algo falla  
    SELECT ERROR_NUMBER() AS numero, ERROR_MESSAGE() AS mensaje;  
END CATCH;
```

### 3.10. Procedimientos almacenados y funciones (básico)

```
-- Procedimiento  
CREATE OR ALTER PROCEDURE dbo.usp_facturas_cliente  
    @cliente_id int  
AS  
BEGIN  
    SELECT * FROM facturas WHERE cliente_id = @cliente_id;  
END;  
GO  
  
EXEC dbo.usp_facturas_cliente @cliente_id = 7;  
  
-- Función escalar  
CREATE OR ALTER FUNCTION dbo.fn_con_iva (@importe decimal(12,2))  
RETURNS decimal(12,2)  
AS  
BEGIN  
    RETURN @importe * 1.21;  
END;  
GO  
  
SELECT dbo.fn_con_iva(100);    -- 121.00
```

### 3.11. Comentarios y utilidades

```
-- comentario de una línea  
/* comentario
```

```

de varias líneas */

GO                                -- separador de lotes (no es SQL, lo entiende
SSMS/sqlcmd)
USE nombre_bd;                   -- cambiar de base de datos
SELECT @@ROWCOUNT;              -- filas afectadas por la última instrucción
SELECT @@VERSION;                -- versión de SQL Server
EXEC sp_help 'dbo.facturas';     -- estructura de una tabla
SELECT * FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_NAME = 'facturas'; --
columnas

```

## 4. Ejercicios resueltos y explicados

**Consejo de estudio:** intenta leer primero el enunciado y pensar qué técnica necesitas. Después comprueba la solución. Cada ejercicio tiene: **Enunciado** → **Cómo pensarlo** → **Solución** → **Explicación** → **Errores típicos**.

Mapa de la base de datos (deducido de los ejercicios)

```

clientes    (cliente_id, razon_social, pais, segmento, rating, limite_credito,
gestor_id → empleados)
facturas    (factura_id, cliente_id → clientes, unidad_id → unidades, cuenta_id →
cuentas,
            fecha_emision, fecha_vencimiento, importe)
cobros      (cobro_id, factura_id → facturas, fecha_cobro, importe, metodo)
gastos      (unidad_id, cuenta_id, fecha, importe)
presupuestos(periodo_id → periodos, unidad_id, cuenta_id, importe)
periodos    (periodo_id, ejercicio, trimestre, fecha_inicio, fecha_fin)
cuentas     (cuenta_id, nombre, tipo)                -- tipo: Ingreso / Gasto
unidades    (unidad_id, nombre, tipo, unidad_padre_id → unidades)
empleados   (empleado_id, nombre, puesto, jefe_id → empleados)
stg_cobros  (cobro_id, factura_id, fecha_cobro, importe, metodo) -- tabla de
staging

```

Es un esquema reconstruido a partir de las consultas; si en tu práctica algún nombre difiere, prevalece el de tu base de datos.

### Pregunta 1 — JOIN

#### Enunciado

El equipo de Datos va a construir la tabla de hechos de ventas internacionales. Antes de automatizar la carga, Tomás te pide comprobar a mano que las facturas cruzan correctamente con sus tres dimensiones: cliente, unidad y cuenta.

Muestra las facturas emitidas en el **T3 de 2026** a clientes de **fuera de España** cuyo importe sea de **al menos 80.000 €**.

Columnas:

- `factura_id`
- `fecha_emision`
- `razon_social`
- `pais`
- `unidad`
- `cuenta`
- `importe`

Orden: de mayor a menor importe.

Cómo pensarlo

1. **Tabla principal:** `facturas` (es lo que se pide listar).
2. **Datos de otras tablas:** razón social y país (en `clientes`), unidad (en `unidades`), cuenta (en `cuentas`) → **tres JOIN**.
3. **Tipo de JOIN:** `INNER`, porque solo queremos facturas que crucen correctamente con sus tres dimensiones.
4. **Filtros** (van en `WHERE`): fechas del T3, `pais <> 'ES'`, `importe >= 80000`.
5. **Orden:** `ORDER BY importe DESC`.

Solución

```
SELECT
    f.factura_id,
    f.fecha_emision,
    c.razon_social,
    c.pais,
    u.nombre AS unidad,
    cu.nombre AS cuenta,
    f.importe
FROM facturas f
INNER JOIN clientes c
    ON c.cliente_id = f.cliente_id
INNER JOIN unidades u
    ON u.unidad_id = f.unidad_id
INNER JOIN cuentas cu
    ON cu.cuenta_id = f.cuenta_id
WHERE f.fecha_emision >= '2026-07-01'
    AND f.fecha_emision <= '2026-09-30'
    AND c.pais <> 'ES'
    AND f.importe >= 80000
ORDER BY f.importe DESC;
```

Qué hay que recordar

```
facturas
  ↓
clientes
unidades
cuentas
```

Y los filtros van en **WHERE**.

## Explicación

- **Alias de tabla** (**f**, **c**, **u**, **cu**): acortan el código y evitan ambigüedad.
- **u.nombre AS unidad y cu.nombre AS cuenta**: tanto **unidades** como **cuentas** tienen una columna **nombre**; sin alias saldrían dos columnas iguales y no cumplirías los nombres pedidos.
- **"Al menos 80.000"** significa **>=** (incluye el 80.000). "Más de" sería **>**.
- **"Fuera de España"** se traduce en **<> 'ES'**.

## Errores típicos

- Poner **>** en vez de **>=**.
- Olvidar el alias de **nombre** → columnas con el mismo nombre o ambigüedad.
- Usar **LEFT JOIN** sin necesidad (aquí no cambiaría el resultado, pero no es lo que se pide).
- Clientes con **país** NULL no aparecen con **<> 'ES'** (NULL no cumple ninguna comparación). Si hiciera falta incluirlos: **AND (c.país <> 'ES' OR c.país IS NULL)**.

---

## Pregunta 2 — Subconsulta escalar

### Enunciado

Antes de publicar la facturación en la capa de consumo, el pipeline debe señalar los valores atípicos para que alguien los revise.

Con una **subconsulta escalar**, lista las facturas cuyo importe supera **el doble del importe medio** de todas las facturas.

Columnas:

- **factura\_id**
- **razon\_social**
- **fecha\_emision**
- **importe**

Orden: de mayor a menor importe.

### Cómo pensarlo

1. "Doble de la media" → necesito **un número** (**AVG(importe) \* 2**) para comparar.
2. No puedo escribir **WHERE importe > AVG(importe) \* 2** (los agregados no se permiten en **WHERE**) → calculo la media en una **subconsulta entre paréntesis**.

3. La razón social está en `clientes` → un `INNER JOIN`.

## Solución

```
SELECT
    f.factura_id,
    c.razon_social,
    f.fecha_emision,
    f.importe
FROM facturas f
INNER JOIN clientes c
    ON c.cliente_id = f.cliente_id
WHERE f.importe > (
    SELECT AVG(importe) * 2
    FROM facturas
)
ORDER BY f.importe DESC;
```

## Clave

La subconsulta devuelve **un único número**:

```
SELECT AVG(importe) * 2
FROM facturas
```

## Explicación

- La subconsulta **no está correlacionada** (no usa nada de la consulta exterior): se calcula **una sola vez**.
- La media se calcula sobre **todas** las facturas (no solo las del T3), porque el enunciado dice "de todas las facturas".
- La subconsulta interna va **sin JOIN** y sin filtros: `FROM facturas` a secas.

## Errores típicos

- `WHERE importe > AVG(importe) * 2` → error: "An aggregate may not appear in the WHERE clause".
- Poner `AVG(importe) / 2` (mitad) en vez de `* 2` (doble).
- Poner la subconsulta sin paréntesis.

**Alternativa con función de ventana** (equivalente):

```
WITH base AS (
    SELECT f.factura_id, c.razon_social, f.fecha_emision, f.importe,
           AVG(f.importe) OVER () AS media
    FROM facturas f
    JOIN clientes c ON c.cliente_id = f.cliente_id
)
SELECT factura_id, razon_social, fecha_emision, importe
```

```
FROM base
WHERE importe > media * 2
ORDER BY importe DESC;
```

Cuidado: aquí la media se calcularía **después del JOIN**; si algún cliente faltara, no coincidiría exactamente. Por eso el enunciado pide la subconsulta escalar.

## Pregunta 3 — Subconsultas anidadas

### Enunciado

Riesgos sospecha que los cobros de la cartera que gestionan los empleados con puesto **Comercial** no están bien conciliados y pide una cifra de control antes de lanzar el proceso de conciliación.

**Sin usar JOIN**, con subconsultas anidadas a tres niveles, obtén por método de cobro:

- **metodo**
- número de **cobros**
- **importe\_cobrado**

Solo se deben considerar facturas de clientes cuyo gestor tiene el puesto **Comercial**.

### Cómo pensarlo

1. Se parte de **cobros** (de ahí salen **metodo** e **importe**).
2. Hay que llegar hasta **empleados** pasando por **facturas** y **clientes**. Cada salto es un **IN**:

```
cobros.factura_id    IN  facturas.factura_id
facturas.cliente_id IN  clientes.cliente_id
clientes.gestor_id  IN  empleados.empleado_id  (puesto = 'Comercial')
```

3. **Escribe de dentro hacia fuera**: primero los empleados Comerciales, luego sus clientes, luego las facturas de esos clientes, luego los cobros.
4. Al final, **GROUP BY metodo** con **COUNT(\*)** y **SUM(importe)**.

### Solución

```
SELECT
  co.metodo,
  COUNT(*) AS cobros,
  SUM(co.importe) AS importe_cobrado
FROM cobros co
WHERE co.factura_id IN (
  SELECT f.factura_id
  FROM facturas f
  WHERE f.cliente_id IN (
    SELECT c.cliente_id
```

```
FROM clientes c
WHERE c.gestor_id IN (
    SELECT e.empleado_id
    FROM empleados e
    WHERE e.puesto = 'Comercial'
)
)
)
GROUP BY co.metodo
ORDER BY importe_cobrado DESC;
```

## Esquema mental

```
cobros
↓
facturas
↓
clientes
↓
empleados
```

## Explicación

- **"Tres niveles"** = tres subconsultas dentro de la consulta principal (facturas, clientes, empleados).
- **"Sin JOIN"**: solo se usa **IN**. Si pones un **JOIN** incumples el enunciado aunque el resultado sea correcto.
- **COUNT(\*)** cuenta cobros por método; **SUM(co.importe)** suma lo cobrado.
- Las agregaciones solo afectan a la consulta exterior; las subconsultas solo devuelven listas de identificadores.

## Errores típicos

- Confundir qué columna enlaza cada nivel (**gestor\_id** de clientes ↔ **empleado\_id** de empleados).
- Olvidar el **GROUP BY** al usar **COUNT/SUM** junto a **metodo**.
- Usar **JOIN** por comodidad.

---

## Pregunta 4 — CTE

### Enunciado

El cálculo «importe cobrado por factura» se va a reutilizar en varios procesos del equipo, así que conviene aislarlo como un paso con nombre.

Define una **CTE** que obtenga el importe cobrado de cada factura y úsala para listar las facturas con **cobro parcial**: han recibido algún cobro, pero no están totalmente cobradas.

Columnas:

- **factura\_id**

- `razon_social`
- `importe`
- `cobrado`
- `pendiente`

Orden: de mayor a menor saldo pendiente y, en caso de empate, por `factura_id`.

## Cómo pensarlo

1. **Paso 1 (la CTE):** una fila por factura con `SUM(importe)` de sus cobros → `GROUP BY factura_id`.
2. **Paso 2 (consulta final):** unir `facturas` + `clientes` + la CTE.
3. **Cbro parcial** = `cobrado > 0 y cobrado < importe`.
4. `pendiente = importe - cobrado` (columna calculada).
5. Orden: `pendiente DESC`, y de desempate `factura_id`.

## Solución

```
WITH cobros_por_factura AS (  
    SELECT  
        factura_id,  
        SUM(importe) AS cobrado  
    FROM cobros  
    GROUP BY factura_id  
)  
SELECT  
    f.factura_id,  
    c.razon_social,  
    f.importe,  
    cpf.cobrado,  
    f.importe - cpf.cobrado AS pendiente  
FROM facturas f  
INNER JOIN clientes c  
    ON c.cliente_id = f.cliente_id  
INNER JOIN cobros_por_factura cpf  
    ON cpf.factura_id = f.factura_id  
WHERE cpf.cobrado > 0  
    AND cpf.cobrado < f.importe  
ORDER BY pendiente DESC, f.factura_id;
```

## Explicación

- La CTE **resume los cobros antes de unirlos**: así cada factura aparece una sola vez (si uniéramos `facturas` con `cobros` directamente, una factura con 3 cobros saldría 3 veces).
- El `INNER JOIN` con la CTE ya descarta las facturas sin cobros; el `cpf.cobrado > 0` es una protección extra.
- `ORDER BY pendiente DESC`: aquí **sí** se puede usar el alias porque el `ORDER BY` se ejecuta después del `SELECT`. En el `WHERE` no se podría.
- Una factura **totalmente cobrada** (`cobrado = importe`) queda fuera por el `<`.

## Errores típicos

- Usar `WHERE pendiente > 0` (alias en el WHERE → error).
- Unir `facturas` con `cobros` sin agrupar antes → filas duplicadas y sumas infladas.
- Olvidar el criterio de desempate `f.factura_id`.

---

## Pregunta 5 — CTE encadenadas

### Enunciado

El equipo comercial quiere saber en qué países y segmentos de cliente se concentró la facturación del T3.

Resuélvelo como un pipeline de **CTE encadenadas**:

1. `facturas_t3`: facturas del T3.
2. `enriquecidas`: añade país y segmento.
3. `por_segmento`: agrupa por país y segmento.

La consulta final debe mostrar las combinaciones que facturaron **200.000 € o más**.

### Cómo pensarlo

Cada paso del enunciado es una CTE y **cada una usa la anterior**:

```
facturas   → facturas_t3 → enriquecidas → por_segmento → SELECT final
(todas)    (filtro T3)   (+ país, seg.)   (agrupa)         (≥ 200.000)
```

### Solución

```
WITH facturas_t3 AS (  
    SELECT *  
    FROM facturas  
    WHERE fecha_emision >= '2026-07-01'  
          AND fecha_emision <= '2026-09-30'  
)  
enriquecidas AS (  
    SELECT  
        f.factura_id,  
        f.importe,  
        c.pais,  
        c.segmento  
    FROM facturas_t3 f  
    INNER JOIN clientes c  
        ON c.cliente_id = f.cliente_id  
)  
por_segmento AS (  
    SELECT  
        pais,  
        segmento,
```

```
        COUNT(*) AS facturas,  
        SUM(importe) AS importe  
    FROM enriquecidas  
    GROUP BY pais, segmento  
)  
SELECT  
    pais,  
    segmento,  
    facturas,  
    importe  
FROM por_segmento  
WHERE importe >= 200000  
ORDER BY importe DESC;
```

## Explicación

- Un único **WITH** y las CTE separadas por **comas** (la última no lleva coma).
- **enriquecidas** lee de **facturas\_t3** (no de **facturas**): así mantiene el filtro del T3.
- El filtro **importe >= 200000** va en el **WHERE** de la consulta final porque **por\_segmento ya está agrupada**. Sería equivalente poner **HAVING SUM(importe) >= 200000** dentro de la CTE **por\_segmento**.
- **COUNT(\*)** cuenta facturas por combinación; **SUM(importe)** el total.

## Errores típicos

- Repetir **WITH** antes de cada CTE.
- Leer de **facturas** en vez de la CTE anterior.
- Poner coma después de la última CTE.

---

## Pregunta 6 — CTE recursiva

### Enunciado

Para la dimensión organizativa del modelo analítico hay que aplanar la jerarquía que guarda **dbo.unidades**, de modo que cada unidad lleve su profundidad y su ruta completa.

Con una **CTE recursiva** que parta de la unidad raíz, devuelve todas las unidades con:

- **unidad\_id**
- **nombre**
- **tipo**
- **nivel**
- **ruta**

La raíz es el nivel **0**.

La ruta debe separar los nombres con **>**.

Orden: por **ruta**.

## Cómo pensarlo

1. **Ancla:** la raíz = la unidad sin padre (`unidad_padre_id IS NULL`), con `nivel = 0` y `ruta = nombre`.
2. **Recursión:** cada unidad cuyo padre ya está en la CTE; `nivel = nivel del padre + 1` y `ruta = ruta del padre + ' > ' + nombre`.
3. Unir las dos partes con `UNION ALL`.
4. Ordenar por `ruta` para que salga en orden jerárquico.

## Solución

```
WITH jerarquia AS (  
    SELECT  
        unidad_id,  
        nombre,  
        tipo,  
        unidad_padre_id,  
        0 AS nivel,  
        CAST(nombre AS nvarchar(1000)) AS ruta  
    FROM unidades  
    WHERE unidad_padre_id IS NULL  
  
    UNION ALL  
  
    SELECT  
        u.unidad_id,  
        u.nombre,  
        u.tipo,  
        u.unidad_padre_id,  
        j.nivel + 1,  
        CAST(j.ruta + N' > ' + u.nombre AS nvarchar(1000))  
    FROM unidades u  
    INNER JOIN jerarquia j  
        ON u.unidad_padre_id = j.unidad_id  
)  
SELECT  
    unidad_id,  
    nombre,  
    tipo,  
    nivel,  
    ruta  
FROM jerarquia  
ORDER BY ruta  
OPTION (MAXRECURSION 100);
```

## Muy importante

En una CTE recursiva existen dos partes:

```
-- Ancla
SELECT ...

UNION ALL

-- Recursión
SELECT ...
```

## Explicación

- **CAST(... AS nvarchar(1000)) en las dos partes:** SQL Server exige que el tipo de cada columna sea **idéntico** en el ancla y en la recursión. Sin el **CAST**, **ruta** en el ancla tendría la longitud de **nombre** y la concatenación de la recursión sería más larga → error *"Types don't match between the anchor and the recursive part"*.
- **N' > ':** la **N** marca el literal como Unicode (**nvarchar**), coherente con el **nvarchar** de la ruta.
- **j.nivel + 1:** **j** es la fila del padre ya calculada; sumamos un nivel.
- **ON u.unidad\_padre\_id = j.unidad\_id:** el hijo apunta a su padre, que ya está en **jerarquia**.
- **OPTION (MAXRECURSION 100)** va **al final de la consulta que usa la CTE** (no dentro de la CTE).

## Errores típicos

- No poner el **CAST** en ambas partes.
- Invertir el **ON** (**u.unidad\_id = j.unidad\_padre\_id**) → recorrería la jerarquía hacia arriba o daría resultados vacíos.
- Usar **UNION** en vez de **UNION ALL** (no se permite en la recursión).
- Poner **OPTION** dentro de la CTE.

---

## Pregunta 7 — Vista

### Enunciado

Hugo Barrios, **Financial Analyst**, monta cada trimestre la cuenta de resultados del grupo en su herramienta de BI. No debe consultar las tablas base ni conocer cómo se cruzan.

Crea la vista:

```
dbo.vw_cuenta_resultados
```

Debe tener:

- **ejercicio**
- **trimestre**
- **tipo**
- **cuenta**
- **importe**

La vista reúne:

- facturas → ingresos
- gastos → gastos

Las facturas se fechan por `fecha_emision` y los gastos por `fecha`.

## Cómo pensarlo

1. "No debe conocer cómo se cruzan las tablas" → **vista**.
2. Hay **dos orígenes** (facturas y gastos) con la misma forma de salida → dos `SELECT` unidos con `UNION ALL`.
3. Para sacar `ejercicio` y `trimestre` hay que cruzar cada fecha con `periodos` (**join por rango**: `fecha BETWEEN fecha_inicio AND fecha_fin`).
4. `tipo` y `cuenta` vienen de `cuentas` (el tipo distingue Ingreso/Gasto).
5. Cada rama agrupa: `SUM(importe)` por ejercicio, trimestre, tipo y cuenta.

## Solución

```
CREATE OR ALTER VIEW dbo.vw_cuenta_resultados
AS

SELECT
    p.ejercicio,
    p.trimestre,
    c.tipo,
    c.nombre AS cuenta,
    SUM(f.importe) AS importe
FROM facturas f
INNER JOIN periodos p
    ON f.fecha_emision BETWEEN p.fecha_inicio AND p.fecha_fin
INNER JOIN cuentas c
    ON c.cuenta_id = f.cuenta_id
GROUP BY
    p.ejercicio,
    p.trimestre,
    c.tipo,
    c.nombre

UNION ALL

SELECT
    p.ejercicio,
    p.trimestre,
    c.tipo,
    c.nombre AS cuenta,
    SUM(g.importe) AS importe
FROM gastos g
INNER JOIN periodos p
    ON g.fecha BETWEEN p.fecha_inicio AND p.fecha_fin
INNER JOIN cuentas c
```

```
ON c.cuenta_id = g.cuenta_id
GROUP BY
    p.ejercicio,
    p.trimestre,
    c.tipo,
    c.nombre;

GO
```

Para comprobarla:

```
SELECT
    trimestre,
    tipo,
    SUM(importe) AS importe
FROM dbo.vw_cuenta_resultados
GROUP BY trimestre, tipo
ORDER BY trimestre, tipo;
```

## Explicación

- **Join por rango:** `periodos` no se une por una clave, sino comprobando que la fecha cae dentro del periodo.
- **UNION ALL:** ambas ramas deben tener **mismas columnas, mismo orden y tipos compatibles**. Los nombres los toma la primera rama (por eso el `AS cuenta / AS importe`).
- **UNION ALL y no UNION:** `UNION` eliminaría filas idénticas, lo que podría borrar importes legítimos repetidos; además es más lento.
- **El GROUP BY se repite en cada rama:** cada `SELECT` es independiente.
- `GO` cierra el lote; `CREATE VIEW` debe ser la primera instrucción de su lote.

## Errores típicos

- Poner `ORDER BY` dentro de la vista (no permitido sin `TOP`).
- Olvidar el alias de `SUM(...)` → "No column name was specified for column 5".
- Distinto número de columnas entre las dos ramas del `UNION ALL`.

---

## Pregunta 8 — Vista + CTE

### Enunciado

Clara Mendiola, **Budget Analyst**, hace el seguimiento del presupuesto de gastos y necesita comparar cada línea presupuestada con lo realmente gastado.

Crea:

```
dbo.vw_presupuesto_gastos
```

Columnas:

- ejercicio
- trimestre
- unidad
- cuenta
- presupuesto
- gasto\_real
- desviacion

`gasto_real` debe ser 0 si no existen gastos.

La desviación es:

```
gasto_real - presupuesto
```

Cómo pensarlo

1. **La tabla base es presupuestos**: cada línea presupuestada debe aparecer, haya gasto o no.
2. El gasto real hay que **resumirlo antes** (por periodo, unidad y cuenta) en una **CTE**.
3. Se une con **LEFT JOIN** (si no hay gasto, queda **NULL** → **COALESCE(..., 0)**).
4. **desviacion** = **COALESCE(gasto,0)** - presupuesto.
5. Solo cuentas de tipo 'Gasto'.

Solución

```
CREATE OR ALTER VIEW dbo.vw_presupuesto_gastos
AS

WITH gasto_real AS (
    SELECT
        p.periodo_id,
        g.unidad_id,
        g.cuenta_id,
        SUM(g.importe) AS gasto_real
    FROM gastos g
    INNER JOIN periodos p
        ON g.fecha BETWEEN p.fecha_inicio AND p.fecha_fin
    GROUP BY
        p.periodo_id,
        g.unidad_id,
        g.cuenta_id
)
SELECT
    p.ejercicio,
    p.trimestre,
    u.nombre AS unidad,
    c.nombre AS cuenta,
    pr.importe AS presupuesto,
```

```

    COALESCE(gr.gasto_real, 0) AS gasto_real,
    COALESCE(gr.gasto_real, 0) - pr.importe AS desviacion
FROM presupuestos pr
INNER JOIN periodos p
    ON p.periodo_id = pr.periodo_id
INNER JOIN unidades u
    ON u.unidad_id = pr.unidad_id
INNER JOIN cuentas c
    ON c.cuenta_id = pr.cuenta_id
LEFT JOIN gasto_real gr
    ON gr.periodo_id = pr.periodo_id
    AND gr.unidad_id = pr.unidad_id
    AND gr.cuenta_id = pr.cuenta_id
WHERE c.tipo = 'Gasto';
GO

```

**Nota de la revisión:** tu versión original calculaba el periodo con una subconsulta `SELECT g.*, p.periodo_id ...`. Funciona si `gastos` no tiene ya una columna `periodo_id`, pero si la tuviera daría error de columna duplicada. La versión de arriba hace lo mismo con un `JOIN` directo, más simple y más segura.

Comprobación:

```

SELECT
    unidad,
    cuenta,
    presupuesto,
    gasto_real,
    desviacion
FROM dbo.vw_presupuesto_gastos
WHERE ejercicio = 2026
    AND trimestre = 3
    AND desviacion > 5000
ORDER BY desviacion DESC;

```

## Explicación

- **¿Por qué resumir el gasto en una CTE y no unir `gastos` directamente?** Si uniéramos `presupuestos` con `gastos` sin agrupar, cada presupuesto se repetiría tantas veces como gastos tuviera y el presupuesto se contaría varias veces (**multiplicación de filas** o *fan-out*). Agrupando antes, hay **una fila de gasto por clave** y el `LEFT JOIN` es 1 a 1.
- **`LEFT JOIN gasto_real`:** mantiene las líneas presupuestadas sin gasto.
- **Unión por tres claves** (`periodo_id`, `unidad_id`, `cuenta_id`): la combinación identifica una línea presupuestaria.
- **`COALESCE(gr.gasto_real, 0)`** se usa dos veces: en `gasto_real` y dentro de la fórmula de `desviacion` (si no, `NULL - número = NULL`).
- Se puede usar `WITH` dentro de una vista: `CREATE VIEW ... AS WITH ... SELECT ...`.

## Errores típicos

- Usar **INNER JOIN** con `gasto_real` → se pierden las líneas sin gasto.
- Olvidar **COALESCE** en la desviación → **NULL**.
- Unir solo por una o dos claves → combinaciones erróneas.

---

## Pregunta 9 — Vista de riesgo

### Enunciado

Iván Calvo, **Risk Analyst**, revisa cada semana la exposición de crédito de los clientes y necesita una única fuente fiable para su informe.

Crea:

```
dbo.vw_riesgo_cliente
```

Debe haber una fila por cliente, **incluidos los clientes sin facturas**.

Columnas:

- `cliente_id`
- `razon_social`
- `rating`
- `limite_credito`
- `exposicion`
- `saldo_vencido`

La exposición es la suma de saldos pendientes.

El saldo vencido corresponde a facturas con saldo pendiente cuya fecha de vencimiento es anterior al **30/09/2026**.

### Cómo pensarlo

1. "Una fila por cliente, incluidos los que no tienen facturas" → la base es **clientes** con **LEFT JOIN**.
2. Necesito el **saldo pendiente de cada factura** = `importe - cobrado`. Para eso:
  - CTE 1 **cobrado**: suma de cobros por factura.
  - CTE 2 **saldo\_factura**: `factura + COALESCE(cobrado, 0)` → **pendiente**.
3. **exposicion** = `SUM(pendiente)` por cliente.
4. **saldo\_vencido** = suma de **pendiente solo de las facturas vencidas y con saldo** → `SUM(CASE WHEN ... THEN pendiente ELSE 0 END)`.
5. **COALESCE(..., 0)** para clientes sin facturas (el **SUM** daría **NULL**).

### Solución

```
CREATE OR ALTER VIEW dbo.vw_riesgo_cliente
AS

WITH cobrado AS (
    SELECT
        factura_id,
        SUM(importe) AS cobrado
    FROM cobros
    GROUP BY factura_id
),
saldo_factura AS (
    SELECT
        f.factura_id,
        f.cliente_id,
        f.fecha_vencimiento,
        f.importe - COALESCE(c.cobrado, 0) AS pendiente
    FROM facturas f
    LEFT JOIN cobrado c
        ON c.factura_id = f.factura_id
)
SELECT
    cl.cliente_id,
    cl.razon_social,
    cl.rating,
    cl.limite_credito,
    COALESCE(SUM(sf.pendiente), 0) AS exposicion,
    COALESCE(
        SUM(
            CASE
                WHEN sf.fecha_vencimiento < '2026-09-30'
                AND sf.pendiente > 0
                THEN sf.pendiente
                ELSE 0
            END
        ),
        0
    ) AS saldo_vencido
FROM clientes cl
LEFT JOIN saldo_factura sf
    ON sf.cliente_id = cl.cliente_id
GROUP BY
    cl.cliente_id,
    cl.razon_social,
    cl.rating,
    cl.limite_credito;

GO
```

Comprobación:

```
SELECT
    razon_social,
    rating,
    limite_credito,
    exposicion,
    saldo_vencido
FROM dbo.vw_riesgo_cliente
WHERE exposicion > limite_credito
ORDER BY exposicion DESC;
```

## Explicación

- **Dos LEFT JOIN** con motivos distintos: el de **saldo\_factura** mantiene facturas sin cobros (pendiente = importe); el de la consulta final mantiene clientes sin facturas.
- **Agregación condicional** (**SUM(CASE ...)**): permite calcular dos totales distintos en la misma consulta sin repetirla.
- **"Anterior al 30/09/2026"** → **<** estricto (el propio 30/09 **no** cuenta como vencido).
- **El GROUP BY incluye todas las columnas no agregadas** del **SELECT**.
- La exposición suma **todas** las facturas con pendiente; las ya cobradas aportan 0.

## Errores típicos

- Empezar por **facturas** en vez de **clientes** → se pierden clientes sin facturas.
- Olvidar el **COALESCE** → clientes sin facturas con **NULL**.
- Usar **<=** en vez de **<** en el vencimiento.
- No exigir **pendiente > 0** en el vencido (podrían colarse saldos negativos por cobros en exceso).

---

## Pregunta 10 — SELECT INTO

### Enunciado

El equipo de BI va a probar un cuadro de mando nuevo y necesita una copia estática de la facturación del T3 para no trabajar sobre **dbo.facturas**.

Con una instrucción **SELECT ... INTO**, crea:

```
dbo.facturas_2026_t3
```

con las facturas emitidas en el T3 de 2026.

Columnas:

- **factura\_id**
- **cliente\_id**
- **fecha\_emision**
- **fecha\_vencimiento**

- importe

El script debe poder ejecutarse varias veces sin dar error.

## Cómo pensarlo

1. "Copia estática" + "SELECT INTO" → tabla física.
2. "Ejecutarse varias veces sin error" → DROP TABLE IF EXISTS antes.
3. Select de las 5 columnas con filtro del T3.

## Solución

```
DROP TABLE IF EXISTS dbo.facturas_2026_t3;

SELECT
    factura_id,
    cliente_id,
    fecha_emision,
    fecha_vencimiento,
    importe
INTO dbo.facturas_2026_t3
FROM facturas
WHERE fecha_emision >= '2026-07-01'
AND fecha_emision <= '2026-09-30';
```

Comprobación:

```
SELECT
    COUNT(*) AS facturas,
    SUM(importe) AS importe_total,
    MIN(fecha_emision) AS primera_emision,
    MAX(fecha_emision) AS ultima_emision
FROM dbo.facturas_2026_t3;
```

## Explicación

- INTO va entre el SELECT y el FROM.
- Sin el DROP TABLE IF EXISTS, la segunda ejecución falla porque la tabla ya existe.
- La comprobación usa COUNT, SUM, MIN, MAX: la primera y la última fecha deben estar dentro del T3 (01/07 – 30/09).

## Errores típicos

- Escribir CREATE TABLE ... AS SELECT (eso es de Oracle/MySQL; en SQL Server es SELECT ... INTO).
- Olvidar el DROP y que el script falle al repetirlo.

---

## Pregunta 11 — SELECT INTO + CASE

## Enunciado

La capa de consumo necesita una tabla de facturas con el cliente ya resuelto y el estado de cobro calculado.

Con **SELECT ... INTO**, crea:

```
dbo.facturas_enriquecidas
```

Columnas:

- **factura\_id**
- **razon\_social**
- **pais**
- **fecha\_emision**
- **importe**
- **cobrado**
- **pendiente**
- **estado\_cobro**

Estados:

- **Sin cobro** → no ha recibido ningún cobro.
- **Cobro parcial** → ha recibido alguno pero queda saldo.
- **Cobrada** → no queda saldo.

## Cómo pensarlo

1. Igual que la Pregunta 4: CTE **cobros\_por\_factura**.
2. **LEFT JOIN** con la CTE: queremos **todas** las facturas, también las sin cobros.
3. **cobrado = COALESCE(cobrado, 0); pendiente = importe - cobrado.**
4. **estado\_cobro** con **CASE**, de lo más específico a lo más general.
5. **INTO dbo.facturas\_enriquecidas + DROP TABLE IF EXISTS** al principio.

## Solución

```
DROP TABLE IF EXISTS dbo.facturas_enriquecidas;

WITH cobros_por_factura AS (
    SELECT
        factura_id,
        SUM(importe) AS cobrado
    FROM cobros
    GROUP BY factura_id
)
SELECT
    f.factura_id,
    c.razon_social,
    c.pais,
    f.fecha_emision,
```

```
f.importe,  
COALESCE(cpf.cobrado, 0) AS cobrado,  
f.importe - COALESCE(cpf.cobrado, 0) AS pendiente,  
CASE  
    WHEN COALESCE(cpf.cobrado, 0) = 0  
        THEN 'Sin cobro'  
    WHEN cpf.cobrado < f.importe  
        THEN 'Cobro parcial'  
    ELSE 'Cobrada'  
END AS estado_cobro  
INTO dbo.facturas_enriquecidas  
FROM facturas f  
INNER JOIN clientes c  
    ON c.cliente_id = f.cliente_id  
LEFT JOIN cobros_por_factura cpf  
    ON cpf.factura_id = f.factura_id;
```

## Explicación

- **Orden del CASE:** el primer **WHEN** que se cumple gana. Primero se descartan las facturas sin cobro (**COALESCE(...)** = 0, que también cubre el **NULL**); después, en el segundo **WHEN**, **cpf.cobrado** ya no puede ser **NULL**; lo que quede y no sea parcial es **Cobrada**.
- **Sintaxis combinada:** primero el **WITH**, luego el **SELECT** con su **INTO**. El **DROP TABLE IF EXISTS** va **antes** del **WITH** y termina en **;** (así el **WITH** empieza una instrucción limpia).
- Los textos de los estados deben coincidir **exactamente** con el enunciado ('Sin cobro', 'Cobro parcial', 'Cobrada').

## Errores típicos

- **INNER JOIN** con la CTE → desaparecen las facturas sin cobros (justo las **Sin cobro**).
- Preguntar **cobrado < importe** antes de tratar el **NULL**.
- Escribir mal los textos de los estados.

---

## Pregunta 12 — Tabla temporal

### Enunciado

Tesorería pide dos cifras sobre los cobros del T3.

Crea la tabla temporal:

```
#cobros_t3
```

con:

- **cobro\_id**
- **razon\_social**
- **fecha\_cobro**

- importe
- metodo

Después:

**Consulta A:** número de cobros e importe por método.

**Consulta B:** los tres clientes con mayor importe cobrado.

Cómo pensarlo

1. cobros no tiene el cliente → hay que pasar por facturas para llegar a clientes: cobros → facturas → clientes.
2. Filtro: fecha\_cobro dentro del T3.
3. Guardar en #cobros\_t3 con SELECT ... INTO #cobros\_t3.
4. Consulta A: GROUP BY metodo con COUNT(\*) y SUM.
5. Consulta B: TOP 3 + GROUP BY razon\_social + ORDER BY SUM DESC.

Solución

```
DROP TABLE IF EXISTS #cobros_t3;

SELECT
    co.cobro_id,
    c.razon_social,
    co.fecha_cobro,
    co.importe,
    co.metodo
INTO #cobros_t3
FROM cobros co
INNER JOIN facturas f
    ON f.factura_id = co.factura_id
INNER JOIN clientes c
    ON c.cliente_id = f.cliente_id
WHERE co.fecha_cobro >= '2026-07-01'
    AND co.fecha_cobro <= '2026-09-30';
```

Consulta A

```
SELECT
    metodo,
    COUNT(*) AS cobros,
    SUM(importe) AS importe_cobrado
FROM #cobros_t3
GROUP BY metodo
ORDER BY importe_cobrado DESC;
```

Consulta B

```
SELECT TOP 3
    razon_social,
    SUM(importe) AS importe_cobrado
FROM #cobros_t3
GROUP BY razon_social
ORDER BY importe_cobrado DESC;
```

## Explicación

- **La fecha filtrada es co.fecha\_cobro**, no la de emisión: se piden los *cobros* del T3.
- Las consultas A y B leen de la **temporal**, no repiten los JOIN: esa es la ventaja de la tabla de trabajo.
- **TOP 3 + ORDER BY ... DESC**: sin **ORDER BY**, **TOP** devolvería 3 filas cualesquiera. Si hubiera empate en el tercer puesto, **TOP 3 WITH TIES** incluiría a todos los empatados.
- **GROUP BY razon\_social**: es lo único que identifica al cliente en la temporal (no se guardó **cliente\_id**). Si dos clientes compartieran razón social se sumarían juntos; en un caso real guardaríamos también **cliente\_id**.
- Las tres instrucciones (**SELECT INTO** y las dos consultas) deben ejecutarse **en la misma sesión**, o la tabla # no existirá.

## Errores típicos

- Intentar unir **cobros** con **clientes** directamente (no comparten columna).
- Olvidar **ORDER BY** en el **TOP 3**.
- Ejecutar la consulta A en otra ventana/sesión → *"Invalid object name '#cobros\_t3'"*.

---

## Pregunta 13 — Tablas temporales + depuración

### Enunciado

El banco ha enviado el lote de cobros de principios de octubre, almacenado en:

```
dbo.stg_cobros
```

En esta pregunta **no se carga nada** en **dbo.cobros**.

Hay que hacerlo en tres pasos:

1. **#lote\_unico**: eliminar filas repetidas.
2. **#lote\_valido**: quedarse con las filas que:
  - no existan todavía en **dbo.cobros**;
  - tengan una factura existente;
  - tengan importe mayor que 0.
3. Hacer dos consultas:
  - Consulta A: número de filas en cada etapa.
  - Consulta B: contenido de **#lote\_valido**.

## Cómo pensarlo

Es un pequeño **proceso de calidad de datos** con embudo:

```
stg_cobros —DISTINCT—► #lote_unico —3 filtros—► #lote_valido
(bruto)                (sin repetidas)          (listo para cargar)
```

- Quitar repetidas → **SELECT DISTINCT**.
- "No existe ya en **cobros**" → **NOT EXISTS** contra **cobros**.
- "Tiene factura existente" → **EXISTS** contra **facturas**.
- "Importe > 0" → condición simple.

## Solución

### Paso 1 — Lote sin duplicados

```
DROP TABLE IF EXISTS #lote_unico;

SELECT DISTINCT
    cobro_id,
    factura_id,
    fecha_cobro,
    importe,
    metodo
INTO #lote_unico
FROM dbo.stg_cobros;
```

### Paso 2 — Lote válido

```
DROP TABLE IF EXISTS #lote_valido;

SELECT
    l.cobro_id,
    l.factura_id,
    l.fecha_cobro,
    l.importe,
    l.metodo
INTO #lote_valido
FROM #lote_unico l
WHERE NOT EXISTS (
    SELECT 1
    FROM dbo.cobros c
    WHERE c.cobro_id = l.cobro_id
)
AND EXISTS (
    SELECT 1
    FROM dbo.facturas f
```

```
WHERE f.factura_id = l.factura_id
)
AND l.importe > 0;
```

## Consulta A

```
SELECT
    'stg_cobros' AS etapa,
    COUNT(*) AS filas
FROM dbo.stg_cobros

UNION ALL

SELECT
    '#lote_unico',
    COUNT(*)
FROM #lote_unico

UNION ALL

SELECT
    '#lote_valido',
    COUNT(*)
FROM #lote_valido

ORDER BY filas DESC;
```

## Consulta B

```
SELECT *
FROM #lote_valido
ORDER BY cobro_id;
```

## Idea importante

Aquí aparecen varias técnicas juntas:

```
DISTINCT
↓
NOT EXISTS
↓
EXISTS
↓
WHERE importe > 0
```

## Explicación

- **DISTINCT** elimina filas **completamente idénticas** (las 5 columnas). Si dos filas tienen el mismo `cobro_id` pero distinto importe, **no** son duplicados para **DISTINCT** y ambas pasarían (se podría detectar con `GROUP BY cobro_id HAVING COUNT(*) > 1`).
- **NOT EXISTS / EXISTS correlacionados**: `c.cobro_id = l.cobro_id` conecta la subconsulta con la fila actual del lote. Usar **NOT EXISTS** en vez de **NOT IN** evita el problema de los **NULL**.
- **Consulta A con UNION ALL**: cada **SELECT** devuelve (`etapa, filas`); los nombres de columna los fija el primero. El **ORDER BY filas DESC** va **solo al final**. Debe verse un embudo decreciente: `bruto ≥ único ≥ válido`.
- La etapa se escribe como texto fijo (`'#lote_unico'`) para etiquetar cada fila.

## Errores típicos

- Usar **NOT IN** con subconsultas que pueden contener **NULL**.
- Poner **ORDER BY** dentro de cada **SELECT** del **UNION ALL**.
- Insertar en `dbo.cobros` (el enunciado dice explícitamente que **no se carga nada**).

---

## Pregunta 14 — CTE recursiva de empleados

### Enunciado

El equipo de Datos va a restringir el acceso a la información financiera: solo podrán consultarla las personas que dependen, **directa o indirectamente**, del CFO, Álvaro Montes.

La consulta debe seguir funcionando aunque se añadan o eliminen niveles de mando.

Hay dos opciones:

- subconsultas;
- CTE recursiva.

Hay que elegir la técnica adecuada y obtener los empleados que dependen del CFO, sin incluir al CFO.

Columnas:

- `nombre`
- `puesto`
- `nivel`

Los empleados que dependen directamente del CFO tienen nivel **1**.

Orden:

```
nivel
nombre
```

### Cómo pensarlo

1. Palabras clave: "**directa o indirectamente**" y "**aunque se añadan o eliminen niveles**" → profundidad desconocida → **CTE recursiva**.
2. **Ancla**: los empleados cuyo **jefe\_id** es el CFO → nivel 1 (el CFO no se incluye porque ya no es "dependiente").
3. **Recursión**: los empleados cuyo jefe ya está en la CTE → **nivel + 1**.
4. Mostrar solo **nombre**, **puesto**, **nivel**, ordenados por **nivel**, **nombre**.

## Solución

La técnica correcta es una **CTE recursiva**, porque la profundidad de la jerarquía puede cambiar.

Una cadena fija de subconsultas tendría que conocer de antemano cuántos niveles existen.

```
WITH jerarquia AS (  
    -- Empleados que dependen directamente del CFO  
    SELECT  
        e.empleado_id,  
        e.nombre,  
        e.puesto,  
        e.jefe_id,  
        1 AS nivel  
    FROM empleados e  
    WHERE e.jefe_id = (  
        SELECT empleado_id  
        FROM empleados  
        WHERE puesto = 'CFO'  
    )  
  
    UNION ALL  
  
    -- Empleados de niveles posteriores  
    SELECT  
        e.empleado_id,  
        e.nombre,  
        e.puesto,  
        e.jefe_id,  
        j.nivel + 1  
    FROM empleados e  
    INNER JOIN jerarquia j  
        ON e.jefe_id = j.empleado_id  
)  
SELECT  
    nombre,  
    puesto,  
    nivel  
FROM jerarquia  
ORDER BY nivel, nombre  
OPTION (MAXRECURSION 100);
```

## Idea para el examen

Si aparece algo como:

```
"directa o indirectamente"
```

```
"jerarquía"
```

```
"niveles"
```

piensa inmediatamente:

```
CTE RECURSIVA
```

## Explicación

- **El ancla usa una subconsulta escalar** para encontrar el `empleado_id` del CFO. Esto exige que **solo haya un CFO**; si hubiera dos, daría *"Subquery returned more than 1 value"*. Más preciso para este enunciado: `WHERE nombre = 'Álvaro Montes'`.
- **El CFO no aparece** porque el ancla arranca en *sus subordinados* (nivel 1), no en él.
- **Recursión:** `e.jefe_id = j.empleado_id` busca los subordinados de cada empleado ya encontrado.
- Funciona con cualquier número de niveles; la subconsulta anidada solo funciona con los niveles que hayas escrito.

Variante equivalente (CFO como nivel 0 y filtrado final)

```
WITH jerarquia AS (  
    SELECT empleado_id, nombre, puesto, 0 AS nivel  
    FROM empleados  
    WHERE nombre = N'Álvaro Montes'  
  
    UNION ALL  
  
    SELECT e.empleado_id, e.nombre, e.puesto, j.nivel + 1  
    FROM empleados e  
    INNER JOIN jerarquia j ON e.jefe_id = j.empleado_id  
)  
SELECT nombre, puesto, nivel  
FROM jerarquia  
WHERE nivel > 0          -- excluye al CFO  
ORDER BY nivel, nombre;
```

## Errores típicos

- Elegir subconsultas anidadas (se rompen si cambia la jerarquía).
- Incluir al CFO en el resultado.
- Empezar el nivel en 0 cuando el enunciado dice que los directos son nivel 1.

---

## Pregunta 15 — Elegir entre vista, SELECT INTO y tabla temporal

## Enunciado

Tesorería quiere conservar la **foto de cobros a cierre del T3**: el importe cobrado a cada cliente hasta el **30/09/2026**.

Requisitos:

1. La foto **no debe cambiar** cuando se carguen cobros nuevos.
2. Debe **seguir existiendo en enero**.
3. Cualquier analista debe poder consultarla desde su sesión.
4. Debe crearse y cargarse en **una sola instrucción**.

Las opciones son:

- vista;
- tabla creada con **SELECT INTO**;
- tabla temporal.

Hay que crear:

```
cierre_cobros_2026_t3
```

Columnas:

- **cliente\_id**
- **razon\_social**
- **importe\_cobrado**
- **fecha\_foto**

## Cómo pensarlo

Primero **elegir técnica** con los requisitos, luego escribirla:

| Requisito                       | Vista            | SELECT INTO | #temporal               |
|---------------------------------|------------------|-------------|-------------------------|
| 1. No cambia con cobros nuevos  | ✗ (se recalcula) | ✓           | ✓                       |
| 2. Sigue existiendo en enero    | ✓                | ✓           | ✗ (muere con la sesión) |
| 3. Otros analistas la consultan | ✓                | ✓           | ✗ (solo mi sesión)      |
| 4. Una sola instrucción         | ✓                | ✓           | ✓                       |

Solo **SELECT INTO** cumple los cuatro.

## Solución

La opción correcta es:

```
SELECT INTO
```

¿Por qué?

### Vista:

No sirve porque una vista vuelve a ejecutar la consulta sobre los datos actuales. Si posteriormente se cargan nuevos cobros, la "foto" cambiaría.

### Tabla temporal:

No sirve porque es temporal y no está pensada para permanecer hasta enero ni para que otros analistas la consulten desde sus sesiones.

### SELECT INTO:

Sí sirve porque crea una tabla física con una **copia estática** de los datos en ese momento.

Además, permite crearla y cargarla en una única instrucción.

## Código

```
DROP TABLE IF EXISTS dbo.cierre_cobros_2026_t3;  -- opcional, para poder repetir
el script

SELECT
    c.cliente_id,
    c.razon_social,
    COALESCE(SUM(co.importe), 0) AS importe_cobrado,
    CAST('2026-09-30' AS date) AS fecha_foto
INTO dbo.cierre_cobros_2026_t3
FROM clientes c
LEFT JOIN facturas f
    ON f.cliente_id = c.cliente_id
LEFT JOIN cobros co
    ON co.factura_id = f.factura_id
    AND co.fecha_cobro <= '2026-09-30'
GROUP BY
    c.cliente_id,
    c.razon_social;
```

Comprobación:

```
SELECT *
FROM dbo.cierre_cobros_2026_t3
ORDER BY importe_cobrado DESC;
```

## Explicación

- **LEFT JOIN desde clientes:** la foto debe incluir a todos los clientes, aunque no hayan cobrado nada (saldrán con 0).
- **La condición `co.fecha_cobro <= '2026-09-30'` va en el ON, no en el WHERE.** Si estuviera en el **WHERE**, los clientes sin cobros (con **NULL**) serían eliminados y el **LEFT JOIN** se comportaría como un **INNER JOIN**.
- **`COALESCE(SUM(...), 0)`:** clientes sin cobros → 0 en lugar de **NULL**.
- **`CAST('2026-09-30' AS date) AS fecha_foto`:** columna constante que documenta a qué fecha corresponde la foto; el **CAST** fija el tipo **date** en la tabla creada.
- La columna constante **no** hace falta en el **GROUP BY**.

## Errores típicos

- Elegir vista porque "es más cómodo": se actualizaría y la foto dejaría de ser foto.
- Poner el filtro de fecha en el **WHERE**.
- Olvidar el **COALESCE**.

# 5. Chuleta rápida para el examen

## ¿Qué técnica utilizo?

| Si el enunciado dice...                  | Piensa en...           |
|------------------------------------------|------------------------|
| "Combina tablas"                         | <b>JOIN</b>            |
| "Solo coincidencias"                     | <b>INNER JOIN</b>      |
| "Todos los clientes aunque no tengan..." | <b>LEFT JOIN</b>       |
| "Mayor que la media"                     | Subconsulta escalar    |
| "Pertenece a una lista"                  | <b>IN</b>              |
| "Existe al menos uno"                    | <b>EXISTS</b>          |
| "No existe ninguno"                      | <b>NOT EXISTS</b>      |
| "Quiero poner nombre a una consulta"     | <b>CTE</b>             |
| "Varias etapas"                          | CTE encadenadas        |
| "Directa o indirectamente"               | CTE recursiva          |
| "Guardar una consulta como objeto"       | <b>VIEW</b>            |
| "Copia física"                           | <b>SELECT INTO</b>     |
| "Solo durante la sesión"                 | <b>#tabla_temporal</b> |
| "Clasificar según condiciones"           | <b>CASE</b>            |
| "Si es NULL, usa otro valor"             | <b>COALESCE</b>        |

| Si el enunciado dice...            | Piensa en...                                                 |
|------------------------------------|--------------------------------------------------------------|
| "Eliminar repetidos"               | <code>DISTINCT</code>                                        |
| "Los N mejores"                    | <code>TOP N ... ORDER BY ... DESC</code>                     |
| "Mejor de cada grupo"              | <code>ROW_NUMBER() OVER (PARTITION BY ...) en una CTE</code> |
| "Unir resultados de dos consultas" | <code>UNION ALL</code>                                       |
| "Lo que está en A pero no en B"    | <code>EXCEPT / NOT EXISTS</code>                             |

## JOIN que debes memorizar

```
FROM A
INNER JOIN B
  ON A.id = B.id
```

→ Solo coincidencias.

```
FROM A
LEFT JOIN B
  ON A.id = B.id
```

→ Todos los de A.

Con `LEFT JOIN`, los filtros sobre B van en el `ON`; si van en el `WHERE`, se pierden los `NULL`.

## Agregados y funciones más usadas (resumen)

```
COUNT(*)  COUNT(col)  COUNT(DISTINCT col)
SUM(col)  AVG(col)    MIN(col)    MAX(col)
STRING_AGG(col, ', ')
CONCAT(a, b)  CONCAT_WS('-', a, b)  LEN(s)  UPPER(s)  LOWER(s)
LEFT(s, n)  RIGHT(s, n)  SUBSTRING(s, ini, len)  CHARINDEX('x', s)
REPLACE(s, 'a', 'b')  TRIM(s)
YEAR(f)  MONTH(f)  DAY(f)  DATEPART(QUARTER, f)
DATEADD(DAY, 30, f)  DATEDIFF(DAY, f1, f2)  EOMONTH(f)  GETDATE()
ROUND(n, 2)  CEILING(n)  FLOOR(n)  ABS(n)
COALESCE(a, 0)  ISNULL(a, 0)  NULLIF(a, 0)  IIF(cond, si, no)
CAST(x AS tipo)  CONVERT(tipo, x, estilo)  TRY_CAST(x AS tipo)
ROW_NUMBER() OVER (PARTITION BY g ORDER BY o)
```

## EXISTS

```
WHERE EXISTS (  
    SELECT 1  
    FROM ...  
    WHERE ...  
)
```

No importa qué devuelve el `SELECT 1`.

Solo pregunta:

¿Existe alguna fila?

---

## CTE

```
WITH nombre AS (  
    SELECT ...  
)  
SELECT *  
FROM nombre;
```

Varias:

```
WITH a AS (...),  
     b AS (SELECT ... FROM a),  
     c AS (SELECT ... FROM b)  
SELECT * FROM c;
```

## CTE recursiva

```
WITH jerarquia AS (  
  
    -- Ancla  
    SELECT ...  
  
    UNION ALL  
  
    -- Recursión  
    SELECT ...  
    FROM tabla t  
    JOIN jerarquia j  
        ON t.padre_id = j.id  
)  
SELECT *
```

```
FROM jerarquia  
OPTION (MAXRECURSION 100);
```

---

## Vista

```
CREATE OR ALTER VIEW dbo.nombre  
AS  
SELECT ...  
GO
```

---

## SELECT INTO

```
DROP TABLE IF EXISTS dbo.nombre;  
  
SELECT ...  
INTO dbo.nombre  
FROM ...;
```

---

## Tabla temporal

```
DROP TABLE IF EXISTS #nombre;  
  
SELECT ...  
INTO #nombre  
FROM ...;
```

---

## NULL

Nunca:

```
WHERE columna = NULL
```

Correcto:

```
WHERE columna IS NULL
```

O:

```
WHERE columna IS NOT NULL
```

Para sustituirlo:

```
COALESCE(columna, 0)
```

---

## Fechas de esta práctica

Fecha de corte:

30/09/2026

T3:

01/07/2026 → 30/09/2026

Consulta típica:

```
WHERE fecha >= '2026-07-01'  
AND fecha <= '2026-09-30'
```

---

## Orden de escritura de una consulta

```
WITH ...                ← CTE (opcional)  
SELECT [DISTINCT] [TOP n] columnas  
[INTO tabla]            ← si es SELECT INTO  
FROM tabla  
[JOIN ... ON ...]  
[WHERE ...]             ← filtra filas  
[GROUP BY ...]  
[HAVING ...]           ← filtra grupos  
[ORDER BY ...]  
[OPTION (MAXRECURSION n)]
```

---

## 6. Errores típicos y cómo arreglarlos

---

| Mensaje / síntoma                                                                                                                    | Causa                                                       | Solución                                                  |
|--------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|-----------------------------------------------------------|
| <i>Column '...' is invalid in the select list because it is not contained in either an aggregate function or the GROUP BY clause</i> | Columna sin agregar y sin <b>GROUP BY</b>                   | Añádela al <b>GROUP BY</b> o agrégala ( <b>MAX(col)</b> ) |
| <i>Invalid column name 'alias' en el <b>WHERE</b></i>                                                                                | Usar un alias del <b>SELECT</b> en el <b>WHERE</b>          | Repite la expresión, o usa una CTE/subconsulta            |
| <i>An aggregate may not appear in the WHERE clause</i>                                                                               | <b>SUM/COUNT/AVG</b> en el <b>WHERE</b>                     | Muévelo a <b>HAVING</b> o a una subconsulta               |
| <i>Ambiguous column name 'cliente_id'</i>                                                                                            | Columna en dos tablas sin prefijo                           | Usa <b>alias.columna</b>                                  |
| <i>Incorrect syntax near the keyword 'WITH'</i>                                                                                      | La instrucción anterior no acaba en <b>;</b>                | Pon <b>;</b> antes del <b>WITH</b>                        |
| <i>'CREATE VIEW' must be the only statement in the batch</i>                                                                         | Falta <b>GO</b>                                             | Añade <b>GO</b> antes (y después)                         |
| <i>There is already an object named '...' in the database</i>                                                                        | La tabla ya existe ( <b>SELECT INTO</b> )                   | <b>DROP TABLE IF EXISTS</b> antes                         |
| <i>Invalid object name '#tabla'</i>                                                                                                  | Otra sesión o la tabla temporal no existe                   | Ejecuta todo en la misma ventana/sesión                   |
| <i>Subquery returned more than 1 value</i>                                                                                           | Subconsulta con <b>=/&gt;</b> devuelve varias filas         | Usa <b>IN</b> , o asegúrate de que devuelve 1 valor       |
| <i>Types don't match between the anchor and the recursive part</i>                                                                   | Distinto tipo/longitud en CTE recursiva                     | <b>CAST(... AS nvarchar(1000))</b> en ambas partes        |
| <i>The statement terminated. Maximum recursion 100...</i>                                                                            | Jerarquía profunda o bucle                                  | <b>OPTION (MAXRECURSION n)</b> o revisa el <b>ON</b>      |
| <i>The ORDER BY clause is invalid in views, CTEs, subqueries...</i>                                                                  | <b>ORDER BY</b> dentro de vista/CTE                         | Quítalo (ordena al consultar) o usa <b>TOP</b>            |
| <i>No column name was specified for column N</i>                                                                                     | Expresión sin alias en vista/CTE/INTO                       | Pon <b>AS alias</b>                                       |
| <i>All queries combined using UNION ... must have an equal number of expressions</i>                                                 | Distinto nº de columnas                                     | Iguala columnas y orden                                   |
| Los totales salen <b>inflados</b>                                                                                                    | Join que multiplica filas ( <b>1→N</b> ) y luego <b>SUM</b> | Agrega <b>antes</b> en una CTE y luego une                |
| Un <b>LEFT JOIN</b> "no deja filas sin coincidencia"                                                                                 | Filtro de la tabla derecha en el <b>WHERE</b>               | Muévelo al <b>ON</b>                                      |
| <b>NOT IN</b> no devuelve nada                                                                                                       | La subconsulta tiene algún <b>NULL</b>                      | Usa <b>NOT EXISTS</b>                                     |

| Mensaje / síntoma                                                   | Causa                                                                    | Solución                            |
|---------------------------------------------------------------------|--------------------------------------------------------------------------|-------------------------------------|
| La media sale entera (1 en vez de 1,5)                              | AVG sobre <code>int</code>                                               | <code>AVG(col * 1.0)</code>         |
| <i>Divide by zero error</i>                                         | Divisor <code>0</code>                                                   | <code>x / NULLIF(divisor, 0)</code> |
| Clientes sin datos salen <code>NULL</code> en vez de <code>0</code> | <code>SUM</code> sin coincidencias                                       | <code>COALESCE(SUM(x), 0)</code>    |
| Faltan facturas del 30/09                                           | Columna <code>datetime</code> con hora y <code>&lt;= '2026-09-30'</code> | <code>&lt; '2026-10-01'</code>      |

## 7. Estrategia y checklist final

Cuando veas un ejercicio en el examen:

### 1. Busca las tablas

Pregúntate:

¿Qué información necesito?  
 ¿En qué tabla está?  
 ¿Cómo se relacionan?

### 2. Busca la palabra clave del enunciado

"coinciden" → `JOIN`  
 "todos" → `LEFT JOIN`  
 "lista" → `IN`  
 "existe" → `EXISTS`  
 "no existe" → `NOT EXISTS`  
 "media" → subconsulta escalar  
 "CTE" → `WITH`  
 "jerarquía" → CTE recursiva  
 "vista" → `CREATE OR ALTER VIEW`  
 "copia" → `SELECT INTO`  
 "temporal" → `#`  
 "foto" / "estática" → `SELECT INTO` (tabla física)  
 "sin usar JOIN" → subconsultas con `IN` / `EXISTS`  
 "repetibles" → `DROP TABLE IF EXISTS / CREATE OR ALTER`

### 3. Haz primero la consulta base

Por ejemplo:

```
SELECT ...
FROM ...
```

JOIN ...

Después añade:

```
WHERE
GROUP BY
HAVING
ORDER BY
```

#### 4. Comprueba siempre

Si el ejercicio da una salida esperada, úsala para detectar errores.

Especialmente comprueba:

- número de filas;
- columnas;
- nombres;
- orden;
- **NULL**;
- duplicados;
- filtros de fechas.

#### 5. Checklist de última hora (antes de entregar)

- ☐ ¿Los nombres de columna coinciden EXACTAMENTE con los pedidos?
- ☐ ¿Están en el ORDEN de columnas pedido?
- ☐ ¿El ORDER BY es el pedido (ASC/DESC, desempates)?
- ☐ ¿"Al menos"/"más de"/"anterior a" → >=, >, < correctos?
- ☐ ¿Fechas: T3 = 01/07 a 30/09 de 2026?
- ☐ ¿LEFT JOIN donde dicen "todos / aunque no tengan"?
- ☐ ¿COALESCE donde puede haber NULL en sumas o restas?
- ☐ ¿GROUP BY con TODAS las columnas no agregadas?
- ☐ ¿Hay un DROP ... IF EXISTS si el script debe repetirse?
- ☐ ¿GO antes/después de CREATE VIEW?
- ☐ ¿Respeto la técnica impuesta (CTE, sin JOIN, SELECT INTO...)?
- ☐ ¿Ejecuté el script entero de arriba abajo sin errores?

#### 6. Si algo no funciona

- 1) Lee el mensaje de error: dice la línea y la causa.
- 2) Ejecuta la consulta por partes (la CTE sola, la subconsulta sola).
- 3) Usa SELECT TOP 10 \* para ver cómo son los datos.
- 4) Revisa alias, comas y paréntesis.

5) Si te bloqueas, simplifica: entrega la versión que funciona aunque sea menos elegante.